

Randomized Algorithms

A complete guide to randomized algorithms for SDE placements, online assessments, and competitive programming.

RANDOMIZED QUICKSORT

1. Overview

Randomized Quicksort is a variation of the classic Quicksort algorithm where the pivot element is chosen randomly instead of using a fixed position (like the first, last, or middle element). This randomization eliminates the worst-case $O(n^2)$ behavior on sorted or nearly-sorted input, making the algorithm run in **expected $O(n \log n)$** time for all inputs.

It is a **Las Vegas** style randomized algorithm — it always produces the correct result, but the running time is a random variable.

2. Intuition

The core idea of Quicksort is divide-and-conquer: pick a pivot, partition the array around it, then recursively sort both halves.

The problem with standard Quicksort (fixed pivot) is that if the input is already sorted or reverse-sorted, the partitions become highly unbalanced — one side has $n-1$ elements, the other has 0. This leads to $O(n^2)$ time.

Analogy: Imagine you are sorting a deck of cards by picking a random card as your reference point. If you always pick the first card, a sorted deck gives you the worst possible split. But if you pick a random card, the chances of repeatedly getting a terrible split are astronomically small.

Why randomization works: When the pivot is chosen uniformly at random, the expected size of the larger partition is at most $3n/4$. This guarantees $O(\log n)$ expected recursion depth and $O(n \log n)$ expected total time.

3. When to Use It

- Sorting large arrays where worst-case $O(n^2)$ is unacceptable
- When you need an in-place sorting algorithm with good cache performance
- When the input distribution is unknown or adversarial
- As a building block for order statistics (quickselect) and other algorithms
- In competitive programming when you need fast sorting and can't risk $O(n^2)$ on edge cases

Common trigger phrases:

- "Sort the array efficiently"
- "Find the kth smallest/largest element" (combined with quickselect)
- "Sort in $O(n \log n)$ expected time"

- "In-place sorting"

4. When Not to Use It

- When $O(n \log n)$ worst-case guarantee is required (use Heap Sort, Merge Sort, or IntroSort)
- When stable sorting is needed (use Merge Sort)
- When sorting linked lists (Merge Sort is better)
- When the data is tiny ($n \leq 20$) — insertion sort is faster
- When recursion depth is a concern on very large arrays (use iterative version or Introsort)
- When random number generation is expensive or unavailable

5. Core Concepts

5.1 Pivot Selection

The pivot is the element around which the array is partitioned. In randomized quicksort, it is chosen uniformly at random from the current subarray.

Why it matters: Random pivot selection ensures that no single input can consistently trigger worst-case behavior.

5.2 Partitioning (Lomuto vs Hoare)

- **Lomuto partition:** Uses the pivot as the last element. Simpler but does more swaps. $O(n)$ swaps.
- **Hoare partition:** Uses two pointers moving toward each other. More efficient (fewer swaps) but slightly trickier to implement correctly.

Why it matters: The partition step is the core operation. Choosing the right partition scheme affects constant factors.

5.3 Expected vs Worst-Case

- **Expected time:** $O(n \log n)$ — the average over all possible random choices.
- **Worst-case time:** $O(n^2)$ — theoretically possible but astronomically unlikely (probability $\leq 2/n!$).

Why it matters: For all practical purposes, randomized quicksort runs in $O(n \log n)$. The probability of $O(n^2)$ is smaller than getting struck by lightning.

5.4 Tail Recursion Optimization

After partitioning, recursively sort the smaller partition first and use tail recursion (or iteration) for the larger one. This guarantees $O(\log n)$ worst-case stack depth.

6. Step-by-Step Algorithm

1. If the array segment has 0 or 1 elements, return (base case).
2. Choose a random index between **low** and **high** (inclusive).
3. Swap the element at the random index with the element at **high** (for Lomuto partition).
4. Partition the array using the pivot (now at **high**):
 - Maintain a pointer **i** for the boundary of elements $\leq$ pivot.

- Iterate j from low to $high-1$.
 - If $arr[j] \leq pivot$, swap $arr[i]$ and $arr[j]$, increment i .
5. Place the pivot in its correct position: swap $arr[i]$ and $arr[high]$.
 6. Recursively sort the left subarray (low to $i-1$) and right subarray ($i+1$ to $high$).

7. Dry Run

Input: $arr = [7, 2, 1, 6, 8, 5, 3, 4]$, $low = 0$, $high = 7$

Step	Action	Array State
Start	Initial array	$[7, 2, 1, 6, 8, 5, 3, 4]$
1	Random pivot index = 6 (value 3)	$[7, 2, 1, 6, 8, 5, 3, 4]$
2	Swap pivot with last (index 7)	$[7, 2, 1, 6, 8, 5, 4, 3]$
3	Partition: $i=0, j=0..6$	
	$j=0: 7 > 3$, no swap	$[7, 2, 1, 6, 8, 5, 4, 3]$
	$j=1: 2 \leq 3$, swap(0,1) $\rightarrow i=1$	$[2, 7, 1, 6, 8, 5, 4, 3]$
	$j=2: 1 \leq 3$, swap(1,2) $\rightarrow i=2$	$[2, 1, 7, 6, 8, 5, 4, 3]$
	$j=3: 6 > 3$, no swap	
	$j=4: 8 > 3$, no swap	
	$j=5: 5 > 3$, no swap	
	$j=6: 4 > 3$, no swap	
4	Place pivot: swap($i=2$, $high=7$)	$[2, 1, 3, 6, 8, 5, 4, 7]$
5	Recurse left: $[2, 1]$, $low=0$, $high=1$	
	Random pivot index = 0 (value 2)	
	Swap(0,1) $\rightarrow [1, 2]$, $i=0, j=0$	
	$j=0: 1 \leq 2$, swap(0,0) $\rightarrow i=1$	
	Place pivot: swap(1,1)	$[1, 2, 3, 6, 8, 5, 4, 7]$
	Left base case (size 0)	
6	Recurse right: $[6, 8, 5, 4, 7]$, $low=3$, $high=7$	
	Random pivot = index 5 (value 5)	
	Swap(5,7) $\rightarrow$ swap $5 \leftrightarrow 7$	$[1, 2, 3, 6, 8, 7, 4, 5]$
	Partition around 5:	
	$j=3: 6 > 5$	
	$j=4: 8 > 5$	

Step	Action	Array State
	j=5: 7 > 5	
	j=6: 4 ≤ 5, swap(3,6) → i=4	[1, 2, 3, 4, 8, 7, 6, 5]
	Place pivot: swap(4,7)	[1, 2, 3, 4, 5, 7, 6, 8]
7	Continue recursion...	
Final	Sorted array	[1, 2, 3, 4, 5, 6, 7, 8]

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

// Partition using Lomuto scheme
int partition(vector<int>& arr, int low, int high) {
    // Pivot is already at 'high' after randomization
    int pivot = arr[high];
    int i = low; // boundary of elements <= pivot

    for (int j = low; j < high; j++) {
        if (arr[j] <= pivot) {
            swap(arr[i], arr[j]);
            i++;
        }
    }
    // Place pivot in correct position
    swap(arr[i], arr[high]);
    return i; // return pivot index
}

// Randomized partition: picks random pivot, swaps to end, then partitions
int randomizedPartition(vector<int>& arr, int low, int high) {
    // Pick a random index between low and high
    int randomIndex = low + rand() % (high - low + 1);
    // Move pivot to the end
    swap(arr[randomIndex], arr[high]);
    return partition(arr, low, high);
}

// Main randomized quicksort function
void randomizedQuickSort(vector<int>& arr, int low, int high) {
    if (low < high) {
        int pivotIndex = randomizedPartition(arr, low, high);

        // Recursively sort elements before and after pivot
        randomizedQuickSort(arr, low, pivotIndex - 1);
        randomizedQuickSort(arr, pivotIndex + 1, high);
    }
}
```

```
// Wrapper function
void sort(vector<int>& arr) {
    srand(time(0)); // seed random number generator
    randomizedQuickSort(arr, 0, arr.size() - 1);
}

// ----- ITERATIVE VERSION (using explicit stack) -----
void iterativeRandomizedQuickSort(vector<int>& arr) {
    srand(time(0));
    stack<pair<int, int>> st;
    st.push({0, (int)arr.size() - 1});

    while (!st.empty()) {
        auto [low, high] = st.top();
        st.pop();

        if (low < high) {
            int pivotIndex = randomizedPartition(arr, low, high);
            // Push larger partition first to limit stack size
            if (pivotIndex - low < high - pivotIndex) {
                st.push({pivotIndex + 1, high});
                st.push({low, pivotIndex - 1});
            } else {
                st.push({low, pivotIndex - 1});
                st.push({pivotIndex + 1, high});
            }
        }
    }
}

// ----- DEMO -----
int main() {
    vector<int> arr = {7, 2, 1, 6, 8, 5, 3, 4};
    sort(arr);
    for (int x : arr) cout << x << " ";
    // Output: 1 2 3 4 5 6 7 8
    return 0;
}
```

9. Python Implementation

```
import random

def partition(arr, low, high):
    """Lomuto partition scheme."""
    pivot = arr[high]
    i = low # boundary of elements <= pivot
    for j in range(low, high):
        if arr[j] <= pivot:
            arr[i], arr[j] = arr[j], arr[i]
```

```

        i += 1
    arr[i], arr[high] = arr[high], arr[i]
    return i

def randomized_partition(arr, low, high):
    """Pick random pivot, then partition."""
    random_index = random.randint(low, high)
    arr[random_index], arr[high] = arr[high], arr[random_index]
    return partition(arr, low, high)

def randomized_quicksort(arr, low, high):
    """Recursive randomized quicksort."""
    if low < high:
        pivot_idx = randomized_partition(arr, low, high)
        randomized_quicksort(arr, low, pivot_idx - 1)
        randomized_quicksort(arr, pivot_idx + 1, high)

def sort(arr):
    randomized_quicksort(arr, 0, len(arr) - 1)

# Demo
arr = [7, 2, 1, 6, 8, 5, 3, 4]
sort(arr)
print(arr)  # [1, 2, 3, 4, 5, 6, 7, 8]

```

10. Code Explanation

`partition(arr, low, high):`

- Takes the last element as pivot (already swapped during randomization).
- `i` tracks the boundary of elements $\leq$ pivot. Initially `i = low`.
- Loop `j` from `low` to `high-1`: if `arr[j] ≤ pivot`, swap it to position `i` and increment `i`.
- After the loop, all elements $\leq$ pivot are at indices `low..i-1`, and elements $>$ pivot are at `i..high-1`.
- Finally, swap pivot (at `high`) with `arr[i]` to place it in its correct sorted position.
- Returns the pivot index `i`.

`randomizedPartition(arr, low, high):`

- Generates a random index in `[low, high]`.
- Swaps the chosen pivot with the last element.
- Calls the standard `partition` function.

`randomizedQuickSort(arr, low, high):`

- Base case: if `low ≥ high`, the segment has 0 or 1 elements.
- Calls `randomizedPartition` to get the pivot index.
- Recursively sorts the left and right subarrays.

Why random pivot matters: Without it, a sorted input would cause `partition` to always return `high`, creating unbalanced recursion of depth n . With randomization, the expected depth is $O(\log n)$.

11. Complexity Analysis

Operation	Complexity	Notes
Expected time	$O(n \log n)$	Average over all random choices
Worst-case time	$O(n^2)$	Probability $\leq 2/n!$ (effectively zero)
Best-case time	$O(n \log n)$	When pivot always splits in middle
Space (recursive)	$O(\log n)$ expected	$O(n)$ worst-case without optimization
Space (iterative)	$O(\log n)$	With explicit stack, push larger partition first
In-place	Yes	Only uses swaps, no extra array
Stable	No	Partition is not stable

12. Common Patterns

Pattern 1: Quickselect (kth smallest/largest)

Identification: "Find the kth smallest/largest element in an array."

Approach: Use randomized partition. If pivot index == k-1, return it. If pivot > k-1, recurse left. Else recurse right.

Example: LeetCode 215 (Kth Largest Element in an Array)

Pattern 2: Sort Colors / Dutch National Flag

Identification: "Sort an array with only 3 distinct values."

Approach: Three-way partition (Dutch flag) instead of standard two-way partition. Handles duplicates efficiently.

Example: LeetCode 75 (Sort Colors)

Pattern 3: Wiggle Sort

Identification: "Rearrange array in alternating order."

Approach: Quickselect to find the median, then use a virtual index mapping to rearrange.

Example: LeetCode 324 (Wiggle Sort II)

13. Common Mistakes

- **Forgetting the random seed:** `srand(time(0))` or `srand(seed)` must be called once. Without it, `rand()` gives the same sequence every run.
- **Picking pivot from wrong range:** `rand() % (high - low + 1) + low` — forgetting the `+ low` or `+1`.
- **Off-by-one in partition:** For Lomuto, the loop should go to `high-1`, not `high`.
- **Infinite recursion:** Forgetting the base case or not updating low/high correctly.

- **Stack overflow:** Without tail recursion optimization, worst-case recursion depth could be $O(n)$. Use iterative version or always recurse on smaller half first.
- **Using `rand()` in C++:** `rand()` has limited range (`RAND_MAX = 32767` on many compilers). Use `mt19937` for serious use.
- **Not handling duplicates:** Lomuto partition handles duplicates fine, but Hoare partition needs careful handling.

14. Edge Cases

- **Empty array:** `low = 0, high = -1` → base case triggers immediately.
- **Single element:** `low = high` → base case.
- **Two elements:** Works fine, simple swap if needed.
- **All equal elements:** Lomuto partition returns `high` every time, making it $O(n^2)$. Use three-way partition (Dutch flag) to handle this.
- **Already sorted:** Random pivot fixes this — no longer a problem.
- **Reverse sorted:** Same as above — random pivot fixes it.
- **Large array (10^6+):** Recursive version may stack overflow. Use iterative version.
- **All elements same value:** With Lomuto, pivot index is always `high`. Three-way partition recommended.

15. Variations

15.1 Three-Way Quicksort (Dutch National Flag)

Partitions into three regions: `< pivot`, `= pivot`, `> pivot`. Handles many duplicates in $O(n \log n)$ or better.

Use: When input has many duplicate values.

15.2 Dual-Pivot Quicksort

Uses two pivots and partitions into three regions. Used in Java's `Arrays.sort()` for primitives. Reduces number of comparisons.

Use: Production-grade sorting, Java's implementation.

15.3 Introsort

Begins with Quicksort, switches to Heap Sort if recursion depth exceeds $O(\log n)$, and switches to Insertion Sort for small subarrays. Used in C++ `std::sort` and Python's `sorted()`.

Use: When $O(n \log n)$ worst-case guarantee is required.

15.4 Quickselect (Randomized Selection)

Uses the same partition step but only recurses on one side. Finds the k th smallest element in expected $O(n)$ time.

Use: Finding median, order statistics.

16. Related Algorithms/Data Structures

Algorithm	Connection	When to Choose
Merge Sort	Also divide-and-conquer, $O(n \log n)$ worst-case	When stable sort needed, or linked list
Heap Sort	$O(n \log n)$ worst-case, in-place	When worst-case guarantee needed, not stable
IntroSort	Hybrid of Quick + Heap + Insertion	General-purpose sorting (C++ <code>std::sort</code>)
Counting Sort	Non-comparison sort, $O(n+k)$	When range of values is small
Quickselect	Uses same partition, only one side	For kth smallest/largest, not full sort

17. Practice Problems

Easy

1. **Sort an Array** — LeetCode 912 (Use any sorting algorithm, implement randomized quicksort)
2. **Kth Largest Element in an Array** — LeetCode 215 (Quickselect approach)

Medium

3. **Sort Colors** — LeetCode 75 (Three-way partition / Dutch flag)
4. **Wiggle Sort II** — LeetCode 324 (Quickselect + virtual indexing)
5. **K Closest Points to Origin** — LeetCode 973 (Quickselect with custom comparator)

Hard

6. **Median of Two Sorted Arrays** — LeetCode 4 (Generalized kth element, can use quickselect idea)
7. **Maximum Swap** — LeetCode 670 (Not directly quicksort, but partition-like logic)
8. **Kth Smallest Element in a Sorted Matrix** — LeetCode 378 (Binary search + counting, but quickselect is alternative)

18. Interview Explanation

"Randomized Quicksort is a divide-and-conquer sorting algorithm that picks a random pivot element to partition the array. The key insight is that by choosing the pivot uniformly at random, the expected time becomes $O(n \log n)$ regardless of the input distribution. The worst case is still $O(n^2)$, but the probability of hitting it is less than $2/n!$ — effectively zero for any practical input. The algorithm works in-place using swaps, making it cache-friendly. I would use the Lomuto partition for simplicity and a random pivot via `rand() % (high - low + 1) + low`. For competitive programming, I'd use the iterative version or Introsort via `std::sort` for guaranteed $O(n \log n)$."

19. Revision Notes

- **Core idea:** Random pivot → expected $O(n \log n)$, worst-case $O(n^2)$ is astronomically unlikely
- **Partition schemes:** Lomuto (simple, more swaps), Hoare (fewer swaps, trickier)
- **Random pivot formula:** `rand() % (high - low + 1) + low`
- **Expected space:** $O(\log n)$ (recursion depth)

- **Not stable, in-place**
- **For duplicates:** Use three-way partition (Dutch flag)
- **Hybrid approach:** Introsort = Quicksort + Heapsort + Insertion sort
- **Quickselect:** Same partition, $O(n)$ expected to find k th element

20. Final Cheat Sheet

When to use	Main operations	Complexity	Key code idea	Edge cases
Sorting large arrays, unknown input distribution	Partition, Recursion	Expected $O(n \log n)$, $O(\log n)$ space	<code>randomIndex = low + rand() % (high - low + 1)</code>	Empty, single, all equal, large n

RESERVOIR SAMPLING

1. Overview

Reservoir Sampling is a family of randomized algorithms for selecting a random sample of k items from a stream of n items, where **n is unknown or too large to fit in memory**. The algorithm makes a single pass over the data and uses $O(k)$ memory, regardless of the stream size.

This is a **Las Vegas** style algorithm — it always produces a valid sample of exactly k items, and the randomness ensures each item has equal probability of being selected.

2. Intuition

Imagine you are standing at the entrance of a stadium with a clipboard. You want to select exactly 10 random people from the crowd as they enter. But you don't know how many people will come — it could be 100, 1000, or 100,000.

The challenge: If you pick the first 10 people, that's biased (early arrivals). If you wait until you know the total count, you need to remember everyone.

Reservoir solution: Fill your clipboard with the first 10 people. Then for each subsequent person, give them a probability of $k / \text{current_count}$ of replacing someone already on your list. This way, at any point, every person seen so far has exactly $k / \text{current_count}$ probability of being in your sample.

Why it works: When the $(i+1)$ th person arrives, they have probability $k/(i+1)$ of entering the reservoir. If they enter, they replace a random existing member. This maintains the invariant that each of the first $i+1$ items has probability $k/(i+1)$ of being in the sample.

3. When to Use It

- When you need a random sample from a data stream or large dataset
- When the total size of data is unknown or infinite
- When memory is limited — cannot store all items
- When you need to make a single pass over the data (online algorithm)

- For distributed/parallel sampling where each partition returns a reservoir
- In database query optimization, A/B testing, and big data pipelines

Common trigger phrases:

- "Random sample of size k from a stream"
- "Select k random elements from a list of unknown size"
- "Reservoir sampling"
- "Single-pass random sampling"
- "Sample without knowing total count"

4. When Not to Use It

- When you know n and the data fits in memory — use `std::sample` or `random.sample` (Fisher-Yates shuffle on indices)
- When you need weighted sampling (use weighted reservoir sampling variation)
- When the stream is small ($n \leq 1000$) — simpler approaches work
- When you need deterministic sampling (reservoir is inherently random)
- When you need to sample without replacement but with non-uniform distribution

5. Core Concepts

5.1 Reservoir (the sample buffer)

An array of size k that holds the current sample. Initially filled with the first k items.

Why it matters: The reservoir is the only memory used. Its size k determines both memory usage and sample quality.

5.2 The Replacement Probability

When processing the i -th item ($i > k$), it is selected with probability k/i . If selected, it replaces a uniformly random item in the reservoir.

Why it matters: This probability is the key to the algorithm's correctness. It ensures that at any point, every item seen so far has equal probability k/i of being in the sample.

5.3 The Invariant

After processing i items, each of the first i items has probability exactly k/i of being in the reservoir.

Why it matters: This invariant is maintained by induction and is the formal proof of correctness.

5.4 Algorithm R (Reservoir Sampling)

The classic algorithm by Alan Waterman (1974). Simple and elegant.

6. Step-by-Step Algorithm

Algorithm R (select k items from a stream of unknown size n):

1. Initialize a reservoir array of size k .

2. Fill the reservoir with the first k items from the stream.
3. For each subsequent item at position i (i starting from $k+1$): a. Generate a random integer j in range $[1, i]$. b. If $j \leq k$, replace `reservoir[j-1]` with the current item.
4. After processing all items, the reservoir contains the random sample.

7. Dry Run

Stream: [A, B, C, D, E, F, G, H], **k = 3**

Step	Item	i	Random j	Action	Reservoir
1	A	1	—	Fill <code>reservoir[0] = A</code>	[A, _, _]
2	B	2	—	Fill <code>reservoir[1] = B</code>	[A, B, _]
3	C	3	—	Fill <code>reservoir[2] = C</code>	[A, B, C]
4	D	4	j=2	$j \leq 3$, replace <code>reservoir[1] = D</code>	[A, D, C]
5	E	5	j=4	$j > 3$, skip	[A, D, C]
6	F	6	j=1	$j \leq 3$, replace <code>reservoir[0] = F</code>	[F, D, C]
7	G	7	j=5	$j > 3$, skip	[F, D, C]
8	H	8	j=3	$j \leq 3$, replace <code>reservoir[2] = H</code>	[F, D, H]

Final sample: [F, D, H] (each item had exactly $3/8$ probability of being selected)

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

// Reservoir sampling: select k random items from a stream
// Returns a vector of size k containing the sample
vector<int> reservoirSample(const vector<int>& stream, int k) {
    int n = stream.size();
    k = min(k, n); // handle case where k > n

    vector<int> reservoir(k);

    // Step 1: Fill reservoir with first k items
    for (int i = 0; i < k; i++) {
        reservoir[i] = stream[i];
    }

    // Step 2: Process remaining items
    for (int i = k; i < n; i++) {
        // Generate random number in [0, i]
        int j = rand() % (i + 1);
        // If j < k, replace reservoir[j] with current item
        if (j < k) {
```

```

        reservoir[j] = stream[i];
    }
}

return reservoir;
}

// ----- WEIGHTED RESERVOIR SAMPLING -----
// A simplified version: A-Chao algorithm
vector<int> weightedReservoirSample(const vector<int>& items,
                                   const vector<double>& weights,
                                   int k) {
    int n = items.size();
    k = min(k, n);

    // Store (item, weight, key) triples
    // key = rand^(1/weight), we keep the top k by key
    vector<tuple<double, int, int>> candidates;

    for (int i = 0; i < n; i++) {
        double key = pow((double)rand() / RAND_MAX, 1.0 / weights[i]);
        candidates.push_back({key, items[i], i});
    }

    // Sort by key descending, take top k
    sort(candidates.rbegin(), candidates.rend());

    vector<int> result;
    for (int i = 0; i < k; i++) {
        result.push_back(get<1>(candidates[i]));
    }
    return result;
}

// ----- DEMO -----
int main() {
    srand(time(0));

    vector<int> stream = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
    int k = 3;

    vector<int> sample = reservoirSample(stream, k);

    cout << "Sample: ";
    for (int x : sample) cout << x << " ";
    cout << endl;

    return 0;
}

```

9. Python Implementation

```

import random

def reservoir_sample(stream, k):
    """Select k random items from a stream using reservoir sampling."""
    reservoir = []

    # Fill reservoir with first k items
    for i, item in enumerate(stream):
        if i < k:
            reservoir.append(item)
        else:
            # Random number in [0, i]
            j = random.randint(0, i)
            if j < k:
                reservoir[j] = item

    return reservoir

# Demo
stream = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
k = 3
sample = reservoir_sample(stream, k)
print(f"Sample: {sample}")

# ----- ONE-LINER (Python 3.6+) -----
# Using random.choices (but this needs the full list)
# import random
# sample = random.sample(stream, k) # needs full list, not a stream

```

10. Code Explanation

Reservoir filling phase (first k items):

- Simply copy the first k items into the reservoir array.
- This is the initial state before any probabilistic decisions.

Processing phase (items i = k to n-1):

- For each item at index *i*, generate a random integer *j* in range $[0, i]$.
- If $j < k$, replace `reservoir[j]` with the current item.
- If $j \geq k$, skip the item.

Why $j < k$ as the condition: The probability of $j < k$ is $k / (i+1)$, which is exactly the probability that the current item should be in the sample.

Why `rand() % (i + 1)`: This gives a uniform random integer in $[0, i]$. The +1 is because `rand() % m` gives a value in $[0, m-1]$.

11. Complexity Analysis

Operation	Complexity	Notes
Time	$O(n)$	Single pass through the stream
Space	$O(k)$	Reservoir of size k only
Preprocessing	$O(k)$	Filling the initial reservoir
Per item	$O(1)$	Constant time: generate random number, maybe swap
Weighted version	$O(n \log n)$	Due to sorting by key

12. Common Patterns

Pattern 1: Random Sample from Data Stream

Identification: "Select k random elements from a stream of unknown size."

Approach: Standard reservoir sampling (Algorithm R).

Example: LeetCode 382 (Linked List Random Node)

Pattern 2: Random Pick with Stream

Identification: "Pick a random element from a stream where each element has equal probability."

Approach: Reservoir sampling with $k=1$. This simplifies to: keep the first item, then for item i , replace with probability $1/i$.

Example: LeetCode 398 (Random Pick Index)

Pattern 3: Weighted Reservoir Sampling

Identification: "Sample from a stream where items have different weights / probabilities."

Approach: A-Chao algorithm: assign each item a key = $\text{random}^{(1/\text{weight})}$, keep top k by key.

Example: Custom A/B testing scenarios.

13. Common Mistakes

- **Forgetting that `rand() % (i+1)` gives range `[0, i]`, not `[1, i+1]`:** The condition `j < k` works correctly with 0-based indexing.
- **Using `j <= k` instead of `j < k`:** This would give probability $(k+1)/(i+1)$ instead of $k/(i+1)$.
- **Modifying the reservoir during iteration over the stream:** The algorithm is designed for single-pass. Don't backtrack.
- **Not handling `k > n`:** If $k > \text{stream size}$, return all items (or throw error).
- **Using reservoir sampling when simple random sampling works:** If the data fits in memory and n is known, use Fisher-Yates shuffle on indices.
- **Not seeding the random number generator:** Without `srand()`, `rand()` gives the same sequence every run.

14. Edge Cases

- **k = 0:** Empty reservoir, return empty vector.
- **k = 1:** Simplifies to "pick one random item from stream." Probability of replacement = $1/i$.
- **k = n:** Reservoir contains all items. No replacements needed.
- **k > n:** Return all items (or handle as error).
- **Empty stream (n = 0):** Return empty reservoir.
- **Stream with 1 item, k = 1:** Reservoir contains that item.
- **Very large stream (n = 10^9):** Algorithm works fine, no overflow issues.

15. Variations

15.1 Algorithm R (k = 1)

Simplifies to: keep the first item; for the i -th item ($i \geq 2$), replace with probability $1/i$.

Use: When you need exactly one random item from a stream.

15.2 Weighted Reservoir Sampling (A-Chao)

Each item has a weight w_i . Items with higher weight are more likely to be selected. Assign each item a key = $\text{random}^{(1/w_i)}$, keep the top k by key.

Use: When items have different importance or probability.

15.3 Distributed Reservoir Sampling

Partition the data across multiple machines. Each machine runs reservoir sampling independently. Then merge reservoirs using weighted selection.

Use: Big data pipelines (MapReduce, Spark, Flink).

15.4 Reservoir Sampling with Deletion

Supports removing items from the sample as the stream progresses. More complex but useful for sliding windows.

16. Related Algorithms/Data Structures

Algorithm	Connection	When to Choose
Fisher-Yates Shuffle	Both produce random samples	Fisher-Yates when n is known, data fits in memory
Random Pick with Weight	Both are random selection	Weighted pick when you need one item with probability proportional to weight
Randomized Quickselect	Both use randomness for selection	Quickselect for order statistics, not sampling
Bloom Filter	Both are probabilistic data structures	Bloom filter for membership, not sampling

17. Practice Problems

Easy

1. **Linked List Random Node** — LeetCode 382 (Reservoir sampling with $k=1$)
2. **Random Pick Index** — LeetCode 398 (Reservoir sampling with $k=1$, handle duplicates)

Medium

3. **Random Pick with Blacklist** — LeetCode 710 (Reservoir sampling variation)
4. **Insert Delete GetRandom O(1)** — LeetCode 380 (Uses array + hash map, not reservoir but related)

Hard

5. **Random Point in Non-overlapping Rectangles** — LeetCode 497 (Weighted random pick, similar to weighted reservoir)
6. **Random Flip Matrix** — LeetCode 519 (Reservoir-like thinking for picking from decreasing pool)

18. Interview Explanation

"Reservoir sampling is used to select k random items from a stream of unknown size using $O(k)$ memory. The algorithm first fills a reservoir with the first k items. Then for each subsequent item at position i , it generates a random number j in $[0, i]$. If $j < k$, it replaces `reservoir[j]` with the current item. The key insight is that this maintains the invariant that each of the first i items has exactly k/i probability of being in the reservoir. The proof is by induction. The time complexity is $O(n)$ for a stream of length n , and space is $O(k)$. I'd use this when I need a random sample from a large dataset that doesn't fit in memory, or when the data arrives as a stream."

19. Revision Notes

- **Core idea:** First k items fill reservoir, then each item i ($i > k$) has probability k/i of entering
- **Condition:** `rand() % (i+1) < k` → replace `reservoir[j]`
- **Space:** $O(k)$ regardless of stream size
- **Time:** $O(n)$ single pass
- **Key invariant:** Each of first i items has probability k/i of being in sample
- **$k=1$ case:** Replace current item with probability $1/i$
- **Not suitable when:** n is known and data fits in memory (use Fisher-Yates)

20. Final Cheat Sheet

When to use	Main operations	Complexity	Key code idea	Edge cases
Random sample from stream, unknown n , $O(k)$ memory	Fill reservoir, replace with probability	Time $O(n)$, Space $O(k)$	<code>if (rand() % (i+1) < k) reservoir[j] = item</code>	$k=0$, $k>n$, empty stream, $k=1$

SHUFFLE ALGORITHM (FISHER-YATES SHUFFLE)

1. Overview

The Fisher-Yates Shuffle (also known as the Knuth Shuffle) is an algorithm for generating a random permutation of a finite sequence. It produces an **unbiased** shuffle — every permutation of the n elements is equally likely. The algorithm runs in **$O(n)$ time** and uses **$O(1)$ extra space** by shuffling in-place.

It is a **Las Vegas** style algorithm — it always produces a valid permutation, and the randomness determines which one.

2. Intuition

Imagine you have a deck of cards and want to shuffle it thoroughly. The simplest way: pick a random card, put it aside, then pick another random card from the remaining ones, and repeat until all cards are moved.

The Fisher-Yates insight: Instead of moving cards to a separate pile (which requires $O(n)$ extra space), you can shuffle in-place. At step i , pick a random card from the remaining unshuffled portion (positions i to $n-1$) and swap it to position i .

Analogy: Think of the array as having two zones: the "shuffled" prefix (indices 0 to $i-1$) and the "unshuffled" suffix (indices i to $n-1$). At each step, you pick a random element from the suffix and move it to the boundary between the two zones.

Why it works: Each element has an equal chance of ending up in any position because at each step, the probability of picking any particular remaining element is exactly $1/(\text{remaining count})$.

3. When to Use It

- When you need a random permutation of an array
- When you need to randomly select k items without replacement (partial shuffle)
- When you need an unbiased, uniform shuffle
- In game development (shuffling decks, tiles, loot tables)
- In randomized algorithms (randomized quicksort, randomized BSTs)
- In A/B testing (randomizing treatment assignment)
- In Monte Carlo simulations (random permutations)

Common trigger phrases:

- "Shuffle the array randomly"
- "Random permutation"
- "Randomize the order"
- "Select k random elements without replacement"
- "Generate a random permutation"

4. When Not to Use It

- When you need a random sample with replacement (use random index selection)
- When you need to shuffle a streaming dataset (use reservoir sampling)
- When the array is read-only (copy first, then shuffle)
- When you need a deterministic "shuffle" (use a seeded random number generator)
- When n is very large and you only need a few random elements (partial shuffle is fine, but reservoir sampling may be better for streams)
- When you need a shuffle that is biased in some specific way (not Fisher-Yates)

5. Core Concepts

5.1 In-Place Shuffling

The algorithm modifies the original array directly, using only swaps. No extra array is needed.

Why it matters: $O(1)$ extra space and excellent cache performance.

5.2 Unbiased / Uniform Distribution

Every permutation of the n elements is equally likely (probability $1/n!$).

Why it matters: Many applications (statistics, games, cryptography) require unbiased randomness. A biased shuffle can produce incorrect results.

5.3 The Two-Zone Technique

The array is divided into a "shuffled" prefix and an "unshuffled" suffix. The boundary moves one step right at each iteration.

Why it matters: This is the key idea that enables in-place shuffling. It's also used in selection algorithms (quickselect).

5.4 Forward vs Backward

- **Forward (classic Fisher-Yates):** Pick from unshuffled prefix, place at position i .
- **Backward (Durstenfeld / Knuth):** Pick from unshuffled suffix, swap to end. More common.

Both are equivalent. The backward version is more natural to implement.

6. Step-by-Step Algorithm

Backward Fisher-Yates (Durstenfeld variant):

1. Start from the last index $i = n-1$.
2. Generate a random index j in range $[0, i]$.
3. Swap $arr[i]$ and $arr[j]$.
4. Decrement i by 1.
5. Repeat steps 2-4 until $i = 0$.

Alternative: Forward Fisher-Yates:

1. Start from the first index $i = 0$.
2. Generate a random index j in range $[i, n-1]$.
3. Swap $arr[i]$ and $arr[j]$.
4. Increment i by 1.
5. Repeat steps 2-4 until $i = n-1$.

7. Dry Run

Array: [1, 2, 3, 4, 5]

Backward version:

Step	i	Random j	Array State
0	4	—	[1, 2, 3, 4, 5]
1	4	j=2	swap(4,2) → [1, 2, 5, 4, 3]
2	3	j=0	swap(3,0) → [4, 2, 5, 1, 3]
3	2	j=1	swap(2,1) → [4, 5, 2, 1, 3]
4	1	j=0	swap(1,0) → [5, 4, 2, 1, 3]
5	0	stop	[5, 4, 2, 1, 3]

Forward version (same random choices):

Step	i	Random j	Array State
0	0	—	[1, 2, 3, 4, 5]
1	0	j=2	swap(0,2) → [3, 2, 1, 4, 5]
2	1	j=4	swap(1,4) → [3, 5, 1, 4, 2]
3	2	j=2	swap(2,2) → [3, 5, 1, 4, 2]
4	3	j=3	swap(3,3) → [3, 5, 1, 4, 2]
5	4	stop	[3, 5, 1, 4, 2]

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

// Fisher-Yates shuffle (backward/Durstenfeld variant)
// Shuffles the array in-place
template<typename T>
void shuffle(vector<T>& arr) {
    int n = arr.size();
    // Start from the last element, go backwards
    for (int i = n - 1; i > 0; i--) {
        // Pick a random index from 0 to i (inclusive)
        int j = rand() % (i + 1);
        // Swap current element with randomly picked element
        swap(arr[i], arr[j]);
    }
}

// Forward Fisher-Yates shuffle
template<typename T>
void shuffleForward(vector<T>& arr) {
    int n = arr.size();
```

```

    for (int i = 0; i < n - 1; i++) {
        // Pick a random index from i to n-1 (inclusive)
        int j = i + rand() % (n - i);
        swap(arr[i], arr[j]);
    }
}

// Partial shuffle: shuffle first k elements only
// (effectively selects k random elements at the front)
template<typename T>
void partialShuffle(vector<T>& arr, int k) {
    int n = arr.size();
    k = min(k, n);
    for (int i = 0; i < k; i++) {
        int j = i + rand() % (n - i);
        swap(arr[i], arr[j]);
    }
}

// Get k random elements without replacement (returns a copy)
template<typename T>
vector<T> getRandomSample(vector<T> arr, int k) {
    partialShuffle(arr, k);
    return vector<T>(arr.begin(), arr.begin() + k);
}

// ----- DEMO -----
int main() {
    srand(time(0));

    vector<int> arr = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};

    shuffle(arr);
    for (int x : arr) cout << x << " ";
    cout << endl;
    // Example output: 3 9 1 7 5 2 10 4 8 6

    // Random sample of 3 elements
    vector<int> arr2 = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
    vector<int> sample = getRandomSample(arr2, 3);
    for (int x : sample) cout << x << " ";
    cout << endl;
    // Example output: 8 2 5

    return 0;
}

```

9. Python Implementation

```
import random
```

```

def shuffle(arr):
    """Fisher-Yates shuffle (in-place, backward variant)."""
    n = len(arr)
    for i in range(n - 1, 0, -1):
        j = random.randint(0, i)
        arr[i], arr[j] = arr[j], arr[i]

def shuffle_forward(arr):
    """Forward Fisher-Yates shuffle."""
    n = len(arr)
    for i in range(n - 1):
        j = random.randint(i, n - 1)
        arr[i], arr[j] = arr[j], arr[i]

def partial_shuffle(arr, k):
    """Shuffle first k elements (in-place)."""
    n = len(arr)
    k = min(k, n)
    for i in range(k):
        j = random.randint(i, n - 1)
        arr[i], arr[j] = arr[j], arr[i]

def random_sample(arr, k):
    """Get k random elements without replacement."""
    arr_copy = arr.copy()
    partial_shuffle(arr_copy, k)
    return arr_copy[:k]

# Demo
arr = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
shuffle(arr)
print(arr) # Example: [3, 9, 1, 7, 5, 2, 10, 4, 8, 6]

# Using built-in
arr2 = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
random.shuffle(arr2) # built-in Fisher-Yates
print(arr2)

# Random sample
sample = random.sample(arr2, 3) # built-in, uses Fisher-Yates internally
print(sample)

```

10. Code Explanation

Backward Fisher-Yates:

- Loop `i` from `n-1` down to `1` (not `0`, as swapping with itself is pointless).
- `j = rand() % (i + 1)` picks a random index in `[0, i]`.
- Swap `arr[i]` and `arr[j]`.
- At each step, position `i` is "locked in" — it will never be touched again.

Forward Fisher-Yates:

- Loop i from 0 to $n-2$.
- $j = i + \text{rand}() \% (n - i)$ picks a random index in $[i, n-1]$.
- Swap $\text{arr}[i]$ and $\text{arr}[j]$.
- At each step, position i is "locked in."

Why both are equivalent: The backward version picks a random element from the unshuffled suffix and moves it to the end. The forward version picks a random element from the unshuffled suffix and moves it to the front. Both produce a uniform random permutation.

Why $\text{rand}() \% (i+1)$ is correct: At step i (backward), there are $i+1$ elements in the unshuffled portion. Each should have equal probability $1/(i+1)$ of being chosen. The modulo operation gives a uniform distribution over $[0, i]$.

11. Complexity Analysis

Operation	Complexity	Notes
Time	$O(n)$	Exactly $n-1$ swaps
Space	$O(1)$	In-place, only a few variables
Random numbers	$n-1$	One per iteration
Per element	$O(1)$	Constant time: swap + random
Partial shuffle (k)	$O(k)$	Only k iterations

12. Common Patterns

Pattern 1: Full Shuffle

Identification: "Shuffle the array randomly / generate a random permutation."

Approach: Standard Fisher-Yates (backward or forward).

Example: LeetCode 384 (Shuffle an Array)

Pattern 2: Random Sample Without Replacement

Identification: "Select k random elements from an array."

Approach: Partial Fisher-Yates: shuffle first k positions, then return first k elements.

Example: LeetCode 398 (Random Pick Index) — but for multiple picks, use reservoir.

Pattern 3: Shuffle with Seeded Random

Identification: "Deterministic shuffle for reproducibility."

Approach: Use a seeded PRNG (e.g., `mt19937` with fixed seed) instead of `rand()`.

Example: Game replays, A/B testing buckets.

13. Common Mistakes

- **Using `rand() % n` with `n` not uniform:** `rand() % n` is biased if `RAND_MAX + 1` is not divisible by `n`. Use `mt19937` and `uniform_int_distribution` for unbiased results.
- **Including the already shuffled elements:** The random index must be chosen from the unshuffled portion only. For backward: `rand() % (i+1)`, not `rand() % n`.
- **Off-by-one in range:** `j = rand() % (i+1)` gives range `[0, i]`. `j = rand() % i` would miss index `i`.
- **Swapping with self unnecessarily:** When `j == i`, the swap does nothing. This is harmless but wastes a random number.
- **Not seeding:** `srand(time(0))` must be called once. Without it, the same "random" sequence appears every run.
- **Shuffling a const array:** The algorithm modifies the array. If you need to preserve the original, copy it first.
- **Using `random_shuffle` (deprecated in C++17):** Use `std::shuffle` with a proper RNG instead.

14. Edge Cases

- **Empty array:** No iterations, array stays empty.
- **Single element (`n=1`):** The loop `for (i = n-1 = 0; i > 0; i--)` doesn't execute. Array stays the same — which is correct (only one permutation).
- **Two elements:** One swap, either original order or swapped. Both equally likely.
- **All equal elements:** Shuffle produces no visible change, but the algorithm runs correctly.
- **Large array (10^6+):** $O(n)$ time, $O(1)$ space. Works fine.
- **Repeated shuffling:** Each shuffle is independent; the array is re-randomized each time.

15. Variations

15.1 Non-Repeating Random Sampling (Partial Shuffle)

Shuffle only the first `k` elements, then return them. This is more efficient than full shuffle when $k \ll n$.

Use: When you need `k` random elements without replacement.

15.2 Sattolo's Algorithm

A variation that produces a random **cyclic permutation** (no fixed points). Each element ends up in a different position.

Use: Generating random derangements.

15.3 Inside-Out Algorithm

A version that can be used when the array size is unknown in advance (similar to reservoir sampling). Builds the shuffled array while reading the input.

Use: When the input is a stream and you need a random permutation.

16. Related Algorithms/Data Structures

Algorithm	Connection	When to Choose
Reservoir Sampling	Both produce random subsets	Reservoir for streams of unknown size, Fisher-Yates for known array
Randomized Quicksort	Uses random permutation	Quicksort uses random pivot, Fisher-Yates can pre-randomize input
std::shuffle	C++ standard library implementation	Use <code>std::shuffle</code> with <code>mt19937</code> for production code
random.sample	Python's built-in sample	Uses Fisher-Yates internally

17. Practice Problems

Easy

1. **Shuffle an Array** — LeetCode 384 (Implement Fisher-Yates with reset and shuffle)
2. **Shuffle String** — LeetCode 1528 (Restore string from shuffled indices)

Medium

3. **Random Pick Index** — LeetCode 398 (Use reservoir sampling / Fisher-Yates)
4. **K Closest Points to Origin** — LeetCode 973 (Can use quickselect with random pivot)

Hard

5. **Random Flip Matrix** — LeetCode 519 (Shuffle-like approach to track remaining cells)
6. **Insert Delete GetRandom O(1) - Duplicates allowed** — LeetCode 381 (Array + hash map)

18. Interview Explanation

"Fisher-Yates shuffle generates a uniform random permutation in $O(n)$ time and $O(1)$ extra space. The algorithm works by iterating from the last element to the first, picking a random element from the unshuffled portion and swapping it to the current position. This ensures each permutation has probability $1/n!$. The key to correctness is that the random index is always chosen from the unshuffled portion — never from the already shuffled portion. I'd use it whenever I need to randomize an array or select a random sample without replacement. For unbiased random numbers, I'd use `mt19937` with `uniform_int_distribution` instead of `rand()`, which has bias and limited range."

19. Revision Notes

- **Core idea:** Pick random from unshuffled suffix, swap to boundary
- **Backward version:** `for i = n-1 down to 1: j = rand() % (i+1); swap(arr[i], arr[j])`
- **Forward version:** `for i = 0 to n-2: j = i + rand() % (n-i); swap(arr[i], arr[j])`
- **Time:** $O(n)$, **Space:** $O(1)$ in-place
- **Unbiased:** Every permutation equally likely ($1/n!$)
- **Common trap:** Picking j from the wrong range (including already shuffled elements)
- **For production:** Use `std::shuffle(arr.begin(), arr.end(), rng)` with `mt19937`
- **Partial shuffle:** First k elements get shuffled, rest stay in place

20. Final Cheat Sheet

When to use	Main operations	Complexity	Key code idea	Edge cases
Random permutation, random sample without replacement	Swap with random from unshuffled portion	Time $O(n)$, Space $O(1)$	<code>j = rand() % (i+1); swap(arr[i], arr[j])</code>	Empty, $n=1$, all equal

RANDOM PICK WITH WEIGHT

1. Overview

Random Pick with Weight (also known as Weighted Random Selection) is a technique for selecting an element from a collection where each element has a **weight** that determines its probability of being selected. Elements with higher weight are more likely to be chosen.

The standard approach uses **prefix sums** and **binary search** to achieve $O(\log n)$ per pick, with $O(n)$ preprocessing. It is a **Monte Carlo** style algorithm — it's randomized, and the probability distribution matches the weights.

2. Intuition

Imagine you have a spinner (like in a board game) divided into sectors of different sizes. The larger the sector, the more likely the spinner lands on it.

The analogy: Each element's weight is the size of its sector. The total weight is the full circle. To pick a random element, you spin the wheel (generate a random number) and see which sector it lands in.

The prefix sum approach: Imagine arranging the weights as adjacent blocks on a number line. Block 1 spans $[0, w_1)$, block 2 spans $[w_1, w_1 + w_2)$, block 3 spans $[w_1 + w_2, w_1 + w_2 + w_3)$, etc. Generate a random number in $[0, \text{totalWeight})$, and find which block it falls into using binary search on the prefix sums.

Why it works: The probability of landing in any block is proportional to its length (weight). Since blocks are adjacency, the total probability sums to 1.

3. When to Use It

- When you need to pick elements with non-uniform probability
- When you have precomputed weights that don't change frequently
- In reinforcement learning (epsilon-greedy, softmax action selection)
- In load balancing (weighted round-robin with random selection)
- In A/B testing with traffic allocation
- In game development (loot tables, weighted random drops)
- In Monte Carlo Tree Search (MCTS) node selection

Common trigger phrases:

- "Pick an element with probability proportional to its weight"

- "Weighted random selection"
- "Random pick with weight"
- "Non-uniform random selection"
- "Probability proportional to value"

4. When Not to Use It

- When weights change frequently (each update requires $O(n)$ prefix sum rebuild)
- When you need to pick from a dynamic set (add/remove elements) — use a Fenwick tree or segment tree
- When you need exactly k elements without replacement (use weighted reservoir sampling)
- When all weights are equal (just use uniform random selection)
- When n is small ($n \leq 10$) — linear scan is simpler and fast enough
- When you need $O(1)$ pick time — use the alias method (Vose's algorithm)

5. Core Concepts

5.1 Prefix Sum Array

An array `pref[i]` stores the sum of weights from index 0 to i . The total weight is `pref[n-1]`.

Why it matters: Prefix sums convert the weighted selection problem into a binary search problem. The i -th element occupies the range `[pref[i-1], pref[i])` in the prefix sum space.

5.2 Binary Search on Prefix Sums

Generate a random number r in `[0, totalWeight)`. Use binary search (`upper_bound`) to find the first index i where `pref[i] > r`.

Why it matters: Binary search gives $O(\log n)$ pick time. Without it, linear scan would be $O(n)$ per pick.

5.3 The Probability Invariant

For element i with weight w_i , the probability of selection is exactly $w_i / \text{totalWeight}$.

Why it matters: This is the correctness condition. The prefix sum approach guarantees this because the random number falls in the i -th block with probability proportional to w_i .

5.4 Alias Method (Alternative)

Another approach: precompute an "alias table" that allows $O(1)$ pick time. More complex but faster for high-frequency picking.

6. Step-by-Step Algorithm

Preprocessing:

1. Compute the prefix sum array `pref` where `pref[i] = sum(weights[0..i])`.
2. The total weight = `pref[n-1]`.

Pick operation:

1. Generate a random integer r in range $[0, \text{totalWeight})$.
2. Use binary search (upper_bound) to find the first index i where $\text{pref}[i] > r$.
3. Return index i .

7. Dry Run

Weights: $[3, 1, 2, 4]$

Preprocessing:

Index	Weight	Prefix Sum
0	3	3
1	1	4
2	2	6
3	4	10

Total weight = 10

Pick operation:

Random r (range $[0,10)$)	Prefix array	Upper bound (first $> r$)	Selected index
2	$[3, 4, 6, 10]$	$\text{pref}[0] = 3 > 2 \rightarrow 0$	0 (weight 3)
3	$[3, 4, 6, 10]$	$\text{pref}[1] = 4 > 3 \rightarrow 1$	1 (weight 1)
5	$[3, 4, 6, 10]$	$\text{pref}[2] = 6 > 5 \rightarrow 2$	2 (weight 2)
9	$[3, 4, 6, 10]$	$\text{pref}[3] = 10 > 9 \rightarrow 3$	3 (weight 4)
0	$[3, 4, 6, 10]$	$\text{pref}[0] = 3 > 0 \rightarrow 0$	0 (weight 3)

Probability check (over 10,000 picks):

- Index 0: weight 3 / 10 = 30%
- Index 1: weight 1 / 10 = 10%
- Index 2: weight 2 / 10 = 20%
- Index 3: weight 4 / 10 = 40%

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

class RandomPickWithWeight {
private:
    vector<int> prefix; // prefix sum array
    int totalWeight;    // sum of all weights
    mt19937 rng;        // Mersenne Twister RNG
};
```

```

public:
    // Constructor: precomputes prefix sums
    RandomPickWithWeight(vector<int>& weights) {
        int n = weights.size();
        prefix.resize(n);
        prefix[0] = weights[0];
        for (int i = 1; i < n; i++) {
            prefix[i] = prefix[i-1] + weights[i];
        }
        totalWeight = prefix[n-1];
        rng = mt19937(chrono::steady_clock::now().time_since_epoch().count());
    }

    // Pick an index with probability proportional to its weight
    int pickIndex() {
        // Generate random number in [0, totalWeight)
        uniform_int_distribution<int> dist(0, totalWeight - 1);
        int r = dist(rng);

        // Binary search: find first index where prefix[index] > r
        auto it = upper_bound(prefix.begin(), prefix.end(), r);
        return it - prefix.begin();
    }

    // Alternative: manual binary search
    int pickIndexManual() {
        uniform_int_distribution<int> dist(0, totalWeight - 1);
        int r = dist(rng);

        int lo = 0, hi = prefix.size() - 1;
        while (lo < hi) {
            int mid = lo + (hi - lo) / 2;
            if (prefix[mid] > r) {
                hi = mid;
            } else {
                lo = mid + 1;
            }
        }
        return lo;
    }
};

// ----- DEMO -----
int main() {
    vector<int> weights = {3, 1, 2, 4};
    RandomPickWithWeight picker(weights);

    // Test distribution
    map<int, int> freq;
    for (int i = 0; i < 10000; i++) {
        freq[picker.pickIndex()]++;
    }
}

```

```

    cout << "Distribution over 10000 picks:" << endl;
    for (auto& [idx, count] : freq) {
        double pct = 100.0 * count / 10000;
        cout << "Index " << idx << " (weight " << weights[idx] << "): "
             << count << " (" << pct << "%)" << endl;
    }
    /*
    Expected output:
    Index 0 (weight 3): ~3000 (30%)
    Index 1 (weight 1): ~1000 (10%)
    Index 2 (weight 2): ~2000 (20%)
    Index 3 (weight 4): ~4000 (40%)
    */

    return 0;
}

```

9. Python Implementation

```

import random
import bisect

class RandomPickWithWeight:
    def __init__(self, weights):
        """Precompute prefix sums."""
        self.prefix = []
        total = 0
        for w in weights:
            total += w
            self.prefix.append(total)
        self.total_weight = total

    def pick_index(self):
        """Pick an index with probability proportional to weight."""
        r = random.randint(0, self.total_weight - 1)
        # Binary search: find first index where prefix[idx] > r
        return bisect.bisect_right(self.prefix, r)

    def pick_index_manual(self):
        """Manual binary search implementation."""
        r = random.randint(0, self.total_weight - 1)
        lo, hi = 0, len(self.prefix) - 1
        while lo < hi:
            mid = (lo + hi) // 2
            if self.prefix[mid] > r:
                hi = mid
            else:
                lo = mid + 1
        return lo

# Demo

```

```

weights = [3, 1, 2, 4]
picker = RandomPickWithWeight(weights)

# Test distribution
freq = {}
for _ in range(10000):
    idx = picker.pick_index()
    freq[idx] = freq.get(idx, 0) + 1

print("Distribution over 10000 picks:")
for idx, count in sorted(freq.items()):
    pct = 100.0 * count / 10000
    print(f"Index {idx} (weight {weights[idx]}): {count} ({pct:.1f}%)")

# Using Python's built-in random.choices
# selections = random.choices(range(len(weights)), weights=weights, k=10000)

```

10. Code Explanation

Constructor:

- Builds the prefix sum array: `prefix[i] = sum(weights[0..i])`.
- Stores total weight = `prefix[n-1]`.
- Seeds a Mersenne Twister RNG (`mt19937`) for better randomness than `rand()`.

`pickIndex()`:

- Generates a random integer `r` in `[0, totalWeight)`.
- `totalWeight` is exclusive, so the range covers all possible positions.
- `upper_bound(prefix.begin(), prefix.end(), r)` returns an iterator to the first element `> r`.
- The difference `it - prefix.begin()` is the selected index.

Why `upper_bound` and not `lower_bound`:

- The i -th element's range is `[prefix[i-1], prefix[i])`.
- `upper_bound` on `r` finds the first index where `prefix[i] > r`.
- If `r` is exactly `prefix[i-1]`, we want index `i-1` (the element whose range starts at `prefix[i-1]`). `upper_bound(-1)` would give index `i-1` correctly.
- Actually, for `upper_bound`, if `r = prefix[i-1]`, then `upper_bound` returns `prefix.begin() + i` (first `> r`). So we get index `i`. But we want index `i-1` when `r = prefix[i-1]`? No — the range for element `i` is `(prefix[i-1], prefix[i])` or `[prefix[i-1], prefix[i])`. Let's clarify:
 - Element 0 occupies `[0, prefix[0]) = [0, 3)`
 - Element 1 occupies `[prefix[0], prefix[1]) = [3, 4)`
 - Element 2 occupies `[prefix[1], prefix[2]) = [4, 6)`
 - Element 3 occupies `[prefix[2], prefix[3]) = [6, 10)`
 - `r` is in `[0, 10)`. If `r = 0`, `upper_bound` finds first `> 0`, which is `prefix[0] = 3` at index 0. Correct.
 - If `r = 3`, `upper_bound` finds first `> 3`, which is `prefix[1] = 4` at index 1. Correct.
 - If `r = 2.999...` but `r` is integer, `r = 2` gives `upper_bound > 2 = prefix[0] = 3` at index 0. Correct.

- Wait, $r = 3$ is in the range of element 1 $[3, 4)$. `upper_bound` on `prefix` with $r = 3$ finds first > 3 , which is `prefix[1] = 4` at index 1. Correct!

11. Complexity Analysis

Operation	Complexity	Notes
Preprocessing	$O(n)$	Building prefix sum array
Pick	$O(\log n)$	Binary search on prefix sums
Space	$O(n)$	Prefix sum array of size n
Alias method preprocessing	$O(n)$	More complex, but $O(1)$ pick
Dynamic weights (update)	$O(n)$	Must rebuild prefix sums

12. Common Patterns

Pattern 1: Random Pick with Weight (Single Pick)

Identification: "Pick one element with probability proportional to its weight."

Approach: Prefix sums + binary search.

Example: LeetCode 528 (Random Pick with Weight)

Pattern 2: Random Point in Rectangle

Identification: "Pick a random point from a set of rectangles, where larger rectangles are more likely."

Approach: Compute area as weight, prefix sum on areas, pick rectangle, then pick random point inside.

Example: LeetCode 497 (Random Point in Non-overlapping Rectangles)

Pattern 3: Weighted Random Sampling (Multiple Items, Without Replacement)

Identification: "Select k elements with probability proportional to weight, without replacement."

Approach: Weighted reservoir sampling (A-Chao algorithm) or repeated pick with weight + removal.

Example: Custom A/B testing, survey sampling.

13. Common Mistakes

- **Using `totalWeight` instead of `totalWeight - 1` as upper bound:** If `totalWeight` is included, the binary search might go out of bounds.
- **Off-by-one in prefix sum indexing:** `prefix[i]` is the sum of weights $0..i$, not $0..i-1$.
- **Using `lower_bound` instead of `upper_bound`:** This can cause off-by-one errors in the selected index.
- **Not handling integer overflow:** `totalWeight` can overflow a 32-bit int if weights are large and n is large. Use `long long`.
- **Rebuilding prefix sums on every pick:** Preprocess once. If weights change, you need to rebuild.

- **Using `rand()` for serious applications:** `rand()` has limited range and bias. Use `mt19937` with `uniform_int_distribution`.
- **Not seeding the RNG:** Without a seed, the same sequence appears every run.

14. Edge Cases

- **Single element:** `prefix = [w]`, `totalWeight = w`. Always returns index 0.
- **All weights zero:** No valid pick possible. Handle as error or return -1.
- **Zero weight for some elements:** Those elements have 0 probability of being selected, which is correct.
- **Large weights (up to 10^9):** Use `long long` for prefix sums.
- **Large number of elements ($n = 10^5$):** $O(n)$ preprocessing, $O(\log n)$ pick. Works fine.
- **Negative weights:** Not allowed in this algorithm. Weights must be non-negative.
- **Floating-point weights:** Multiply to make them integers, or use a different approach (floating-point prefix sums).

15. Variations

15.1 Alias Method (Vose's Algorithm)

Precomputes an alias table in $O(n)$ time, allowing $O(1)$ pick time. More complex but much faster for high-frequency picking.

Use: When you need millions of picks per second (e.g., game loot tables, Monte Carlo simulations).

15.2 Dynamic Weights (Fenwick Tree)

Use a Fenwick tree (Binary Indexed Tree) to support weight updates and picks in $O(\log n)$ time.

Use: When weights change frequently.

15.3 Weighted Random Sampling Without Replacement

Pick k items from n items, each with probability proportional to weight, without replacement.

Use: Survey sampling, A/B testing with multiple variants.

15.4 Softmax Selection

In reinforcement learning, convert Q-values to probabilities using softmax, then pick using weighted random selection.

Use: Epsilon-greedy with temperature, MCTS.

16. Related Algorithms/Data Structures

Algorithm	Connection	When to Choose
Prefix Sum	Core data structure for this algorithm	General-purpose range sum queries
Binary Search	Used for $O(\log n)$ pick	Standard for sorted arrays
Fenwick Tree	Dynamic version of prefix sums	When weights change frequently

Algorithm	Connection	When to Choose
Segment Tree	More powerful dynamic version	When you need range queries + updates
Reservoir Sampling	Both are random selection	Reservoir for streams with unknown size

17. Practice Problems

Easy

1. **Random Pick with Weight** — LeetCode 528 (Standard implementation)
2. **Random Pick with Blacklist** — LeetCode 710 (Prefix sum + binary search on valid indices)

Medium

3. **Random Point in Non-overlapping Rectangles** — LeetCode 497 (Weighted pick by area)
4. **Random Pick Index** — LeetCode 398 (Uniform pick among duplicates, reservoir sampling)

Hard

5. **Random Flip Matrix** — LeetCode 519 (Track remaining cells, similar to pick with weight)
6. **Random Pick with Weight (Dynamic)** — Not a standard problem, but ask in interviews: "How would you handle weight updates?"

18. Interview Explanation

"Random Pick with Weight selects elements with probability proportional to their weights. I'd use prefix sums and binary search for $O(\log n)$ per pick. First, I compute a prefix sum array where `prefix[i]` is the sum of weights from 0 to i. Then to pick, I generate a random number `r` in $[0, \text{totalWeight})$ and use binary search to find the first index where `prefix[i] > r`. This gives the correct weighted distribution. For static weights, this is optimal. If weights change frequently, I'd use a Fenwick tree to support $O(\log n)$ updates. For maximum speed, I'd use the Alias method for $O(1)$ picks."

19. Revision Notes

- **Core idea:** Prefix sums + binary search for $O(\log n)$ weighted random pick
- **Preprocessing:** `prefix[i] = prefix[i-1] + weights[i]`
- **Pick:** `r = rand() % totalWeight; idx = upper_bound(prefix, r)`
- **Time:** $O(n)$ preprocessing, $O(\log n)$ per pick
- **Space:** $O(n)$
- **Probability:** Each element `i` picked with probability `weights[i] / totalWeight`
- **Common trap:** Off-by-one in prefix sum indexing, integer overflow
- **For dynamic weights:** Use Fenwick tree ($O(\log n)$ update and pick)

20. Final Cheat Sheet

When to use	Main operations	Complexity	Key code idea	Edge cases
Weighted random selection, non-uniform probabilities	Prefix sum, Binary search, Pick	Prep $O(n)$, Pick $O(\log n)$	<code>r = rand() % total; idx = upper_bound(prefix, r)</code>	Zero weights, single element, large weights, overflow

TREAP (RANDOMIZED BINARY SEARCH TREE)

1. Overview

A Treap is a randomized binary search tree that combines a **Binary Search Tree (BST)** with a **Binary Heap**. Each node stores a key (for BST property) and a randomly assigned priority (for heap property). The tree is shaped so that it satisfies both:

- **BST property:** For any node, all keys in the left subtree are smaller, and all keys in the right subtree are larger.
- **Heap property:** The priority of any node is greater than (or less than) the priorities of its children.

The random priorities ensure that the tree is **balanced in expectation**, with $O(\log n)$ expected height for all operations. It is a **Las Vegas** style randomized algorithm.

2. Intuition

Imagine you are building a BST by inserting nodes in random order. If the insertion order is random, the tree is balanced on average — $O(\log n)$ height. But if the insertion order is adversarial (e.g., sorted order), the tree degenerates to a linked list — $O(n)$ height.

The Treap solution: Instead of relying on the insertion order being random, Treap assigns a random priority to each node at creation time. Insertions and deletions use **rotations** (like in AVL trees) to maintain the heap property, which keeps the tree balanced.

Analogy: Think of each node as having a "rank" (priority) assigned randomly. The node with the highest priority becomes the root. The next highest becomes a child of the root, and so on. This is exactly like building a Cartesian tree over the priorities, which gives $O(\log n)$ expected height.

Why it works: The random priorities make the tree **equivalent to a BST built by inserting nodes in random order of priorities**. This is known to have $O(\log n)$ expected height.

3. When to Use It

- When you need a balanced BST with simple implementation
- When you need fast insertion, deletion, and search ($O(\log n)$ expected)
- When you need **order statistics** (kth smallest, rank of a key) — easily added
- When you need range queries or split/merge operations
- In competitive programming as a simpler alternative to AVL or Red-Black trees
- When you need to implement a balanced BST quickly (Treap is much shorter to code than AVL or RBT)

Common trigger phrases:

- "Ordered set / map with order statistics"
- "Kth smallest in a dynamic set"
- "Range sum with insertions/deletions"
- "Split and merge sequences"
- "Implicit treap" (treap on array indices)

4. When Not to Use It

- When you need $O(\log n)$ worst-case guarantees (use AVL, Red-Black, or B-tree)
- When the tree is very small ($n \leq 10$) — simple array operations are faster
- When you need a persistent data structure (though treap can be made persistent easily)
- When you don't need the extra features (order statistics, split/merge) — `std::set` is simpler
- When you need optimal constant factors — AVL trees are more compact
- When random number generation is expensive or unavailable

5. Core Concepts

5.1 Node Structure

Each node has a key, a priority (random), and left/right child pointers.

Why it matters: The priority is the key to the tree's balance. It's assigned once at creation and never changes.

5.2 BST Property

Left subtree < Node < Right subtree (by key).

Why it matters: Enables searching, insertion, and deletion in $O(\text{height})$ time.

5.3 Heap Property

Parent priority > Child priorities (max-heap). Or Parent priority < Child priorities (min-heap).

Why it matters: This is what keeps the tree balanced. The random priorities, combined with the heap property, give the tree its expected $O(\log n)$ height.

5.4 Rotations

Left rotation and right rotation are used to restore the heap property after insertion or deletion.

Why it matters: Rotations change the tree structure without violating the BST property. They are the "tools" that maintain the treap invariant.

5.5 Split and Merge

- **Split:** Split a treap into two treaps based on a key (all keys $\leq k$ go left, $> k$ go right).
- **Merge:** Merge two treaps where all keys in the first are $\leq$ all keys in the second.

Why it matters: Split and merge are the fundamental operations of an implicit treap and make many operations elegant.

6. Step-by-Step Algorithm

Insertion:

- 1. Create a new node with the given key and a random priority.
- 2. Insert the node as in a standard BST (find the correct position).
- 3. While the new node's parent has a lower priority (max-heap), rotate upward.

Alternative (split-based insertion):

- 1. Split the treap into two treaps: left ($keys \leq k$) and right ($keys > k$).
- 2. Merge left, the new node, and right.

Deletion:

- 1. Find the node to delete.
- 2. Rotate it down until it becomes a leaf (by always rotating with the child with higher priority).
- 3. Remove the leaf node.

Alternative (merge-based deletion):

- 1. Find the node to delete.
- 2. Merge its left and right subtrees.
- 3. Attach the merged tree to the parent.

Search: Standard BST search (compare keys, go left or right).

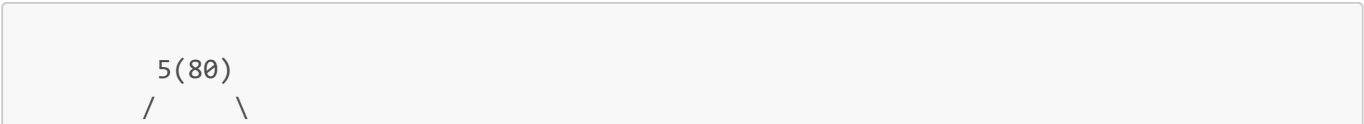
Kth smallest: Each node stores subtree size. Compare k with left subtree size to decide direction.

7. Dry Run

Insert keys: [10, 5, 15, 3, 7] with random priorities [50, 80, 30, 40, 20]

Step	Key	Priority	Action	Tree State
1	10	50	Insert as root	10(50)
2	5	80	Insert as left child of 10. Priority 80 > parent 50 → rotate right	5(80) → right child 10(50)
3	15	30	Insert as right child of 10. Priority 30 < parent 50 → no rotation	5(80) → right 10(50) → right 15(30)
4	3	40	Insert as left child of 5. Priority 40 < parent 80 → no rotation	5(80) → left 3(40), right 10(50) → right 15(30)
5	7	20	Insert as right child of 5. Priority 20 < parent 80 → no rotation	5(80) → left 3(40), right 10(50) → right 15(30); 7 goes between 5 and 10

Final tree:



```

      3(40)  10(50)
           \
          15(30)

```

(Note: 7 would be inserted as right child of 5, but since $7 > 5$, it goes to the right subtree of 5, which is 10. Since $7 < 10$, it becomes left child of 10.)

Corrected final tree:

```

      5(80)
     /  \
   3(40) 10(50)
        /
      7(20)

```

8. C++ Implementation

```

#include <bits/stdc++.h>
using namespace std;

// Generic Treap Node
template<typename T>
struct TreapNode {
    T key;
    int priority;
    int size;           // subtree size (for order statistics)
    TreapNode *left, *right;

    TreapNode(T key) : key(key), priority(rand()), size(1), left(nullptr),
right(nullptr) {}
};

// Update subtree size
template<typename T>
int getSize(TreapNode<T>* node) {
    return node ? node->size : 0;
}

template<typename T>
void updateSize(TreapNode<T>* node) {
    if (node) {
        node->size = 1 + getSize(node->left) + getSize(node->right);
    }
}

// Split by key: first treap = keys ≤ k, second treap = keys > k
template<typename T>
void split(TreapNode<T>* root, T key, TreapNode<T>*& left, TreapNode<T>*& right) {
    if (!root) {

```

```

        left = right = nullptr;
        return;
    }
    if (root->key <= key) {
        // Root and its left subtree go to left treap
        // Split the right subtree
        left = root;
        split(root->right, key, left->right, right);
        updateSize(left);
    } else {
        // Root and its right subtree go to right treap
        // Split the left subtree
        right = root;
        split(root->left, key, left, right->left);
        updateSize(right);
    }
}

// Merge two treaps (all keys in left ≤ all keys in right)
template<typename T>
TreapNode<T>* merge(TreapNode<T>* left, TreapNode<T>* right) {
    if (!left || !right) return left ? left : right;

    if (left->priority > right->priority) {
        // Left becomes root
        left->right = merge(left->right, right);
        updateSize(left);
        return left;
    } else {
        // Right becomes root
        right->left = merge(left, right->left);
        updateSize(right);
        return right;
    }
}

// Insert a key
template<typename T>
TreapNode<T>* insert(TreapNode<T>* root, T key) {
    if (!root) return new TreapNode<T>(key);

    if (key < root->key) {
        root->left = insert(root->left, key);
        if (root->left->priority > root->priority) {
            root = rotateRight(root);
        }
    } else if (key > root->key) {
        root->right = insert(root->right, key);
        if (root->right->priority > root->priority) {
            root = rotateLeft(root);
        }
    }

    // If key == root->key, do nothing (no duplicates)
    // Or handle duplicates by adding a count field
}

```

```

    updateSize(root);
    return root;
}

// Insert using split + merge (alternative)
template<typename T>
TreapNode<T>* insertSplitMerge(TreapNode<T>* root, T key) {
    TreapNode<T>* left;
    TreapNode<T>* right;
    split(root, key, left, right);

    // Check if key already exists (optional)
    TreapNode<T>* mid;
    split(left, key - 1, left, mid); // keys  $\leq$  key-1, then key

    if (mid) {
        // Key already exists, merge back
        return merge(merge(left, mid), right);
    }

    // Create new node and merge
    return merge(merge(left, new TreapNode<T>(key)), right);
}

// Delete a key
template<typename T>
TreapNode<T>* erase(TreapNode<T>* root, T key) {
    if (!root) return nullptr;

    if (key < root->key) {
        root->left = erase(root->left, key);
    } else if (key > root->key) {
        root->right = erase(root->right, key);
    } else {
        // Found the node to delete
        // Merge children and return
        TreapNode<T>* temp = merge(root->left, root->right);
        delete root;
        return temp;
    }

    updateSize(root);
    return root;
}

// Rotations
template<typename T>
TreapNode<T>* rotateRight(TreapNode<T>* node) {
    TreapNode<T>* leftChild = node->left;
    node->left = leftChild->right;
    leftChild->right = node;
    updateSize(node);
    updateSize(leftChild);
}

```

```

        return leftChild;
    }

template<typename T>
TreapNode<T>* rotateLeft(TreapNode<T>* node) {
    TreapNode<T>* rightChild = node->right;
    node->right = rightChild->left;
    rightChild->left = node;
    updateSize(node);
    updateSize(rightChild);
    return rightChild;
}

// Search
template<typename T>
bool search(TreapNode<T>* root, T key) {
    if (!root) return false;
    if (key == root->key) return true;
    if (key < root->key) return search(root->left, key);
    return search(root->right, key);
}

// Kth smallest (1-indexed)
template<typename T>
T kthSmallest(TreapNode<T>* root, int k) {
    if (!root || k < 1 || k > root->size) return -1; // or throw

    int leftSize = getSize(root->left);
    if (k <= leftSize) return kthSmallest(root->left, k);
    if (k == leftSize + 1) return root->key;
    return kthSmallest(root->right, k - leftSize - 1);
}

// Count of elements  $\leq$  key (rank)
template<typename T>
int countLessOrEqual(TreapNode<T>* root, T key) {
    if (!root) return 0;
    if (root->key <= key) {
        return 1 + getSize(root->left) + countLessOrEqual(root->right, key);
    }
    return countLessOrEqual(root->left, key);
}

// Inorder traversal (for debugging)
template<typename T>
void inorder(TreapNode<T>* root) {
    if (!root) return;
    inorder(root->left);
    cout << root->key << "(" << root->priority << ")" ";
    inorder(root->right);
}

// ----- DEMO -----
int main() {

```

```

srand(time(0));

TreapNode<int>* root = nullptr;

// Insert
for (int x : {10, 5, 15, 3, 7, 12, 20}) {
    root = insert(root, x);
}

cout << "Inorder: ";
inorder(root); // 3 5 7 10 12 15 20
cout << endl;

cout << "Search 7: " << (search(root, 7) ? "Found" : "Not found") << endl;
cout << "Search 8: " << (search(root, 8) ? "Found" : "Not found") << endl;

cout << "3rd smallest: " << kthSmallest(root, 3) << endl; // 7
cout << "5th smallest: " << kthSmallest(root, 5) << endl; // 12

root = erase(root, 10);
cout << "After deleting 10: ";
inorder(root); // 3 5 7 12 15 20
cout << endl;

cout << "Size: " << getSize(root) << endl; // 6

return 0;
}

```

9. Python Implementation

```

import random

class TreapNode:
    def __init__(self, key):
        self.key = key
        self.priority = random.randint(1, 2**30)
        self.size = 1
        self.left = None
        self.right = None

def get_size(node):
    return node.size if node else 0

def update_size(node):
    if node:
        node.size = 1 + get_size(node.left) + get_size(node.right)

def split(root, key):
    """Split by key: left = keys ≤ key, right = keys > key."""
    if not root:

```

```

        return None, None
    if root.key <= key:
        left, right = split(root.right, key)
        root.right = left
        update_size(root)
        return root, right
    else:
        left, right = split(root.left, key)
        root.left = right
        update_size(root)
        return left, root

def merge(left, right):
    """Merge two treaps (all keys in left ≤ all keys in right)."""
    if not left or not right:
        return left or right
    if left.priority > right.priority:
        left.right = merge(left.right, right)
        update_size(left)
        return left
    else:
        right.left = merge(left, right.left)
        update_size(right)
        return right

def insert(root, key):
    """Insert a key using split+merge."""
    left, right = split(root, key)
    # Check if key already exists
    left, mid = split(left, key - 1)
    if mid:
        # Key exists, merge back
        return merge(merge(left, mid), right)
    return merge(merge(left, TreapNode(key)), right)

def erase(root, key):
    """Delete a key using split+merge."""
    left, right = split(root, key)
    left, mid = split(left, key - 1)
    # mid contains the node with key (if exists)
    return merge(left, right)

def search(root, key):
    """Search for a key."""
    if not root:
        return False
    if key == root.key:
        return True
    if key < root.key:
        return search(root.left, key)
    return search(root.right, key)

def kth_smallest(root, k):
    """Get kth smallest element (1-indexed)."""

```

```

    if not root or k < 1 or k > root.size:
        return None
    left_size = get_size(root.left)
    if k <= left_size:
        return kth_smallest(root.left, k)
    if k == left_size + 1:
        return root.key
    return kth_smallest(root.right, k - left_size - 1)

def inorder(root):
    if not root:
        return []
    return inorder(root.left) + [root.key] + inorder(root.right)

# Demo
root = None
for x in [10, 5, 15, 3, 7, 12, 20]:
    root = insert(root, x)

print("Inorder:", inorder(root)) # [3, 5, 7, 10, 12, 15, 20]
print("Search 7:", search(root, 7))
print("Search 8:", search(root, 8))
print("3rd smallest:", kth_smallest(root, 3)) # 7
print("5th smallest:", kth_smallest(root, 5)) # 12

root = erase(root, 10)
print("After deleting 10:", inorder(root)) # [3, 5, 7, 12, 15, 20]
print("Size:", root.size if root else 0) # 6

```

10. Code Explanation

Node structure:

- **key**: The value stored in the node (BST property).
- **priority**: Randomly assigned at creation (heap property).
- **size**: Subtree size, used for order statistics.
- **left, right**: Child pointers.

split(root, key):

- If root is nullptr, return (null, null).
- If root->key ≤ key, root and its left subtree belong to the left treap. Recursively split the right subtree.
- If root->key > key, root and its right subtree belong to the right treap. Recursively split the left subtree.
- Updates sizes after splitting.

merge(left, right):

- If either is null, return the other.
- If left->priority > right->priority, left becomes the root. Merge left->right with right.
- Otherwise, right becomes the root. Merge left with right->left.
- Updates sizes after merging.

insert(root, key) (rotation-based):

- Standard BST insertion.
- After insertion, if the child's priority exceeds the parent's, rotate upward.
- This maintains the heap property.

insert(root, key) (split-merge based):

- Split the treap into left ($\text{keys} \leq \text{key}$) and right ($\text{keys} > \text{key}$).
- Split left again into left2 ($\text{keys} \leq \text{key}-1$) and mid (key).
- If mid exists, key is a duplicate. Merge everything back.
- Otherwise, create a new node and merge: `merge(merge(left2, new_node), right)`.

erase(root, key) (merge-based):

- Find the node to delete via BST search.
- When found, merge its left and right children, and return the result.
- This effectively removes the node.

kthSmallest(root, k):

- Uses the **size** field to determine which subtree contains the kth element.
- If $k \leq \text{left_size}$, go left.
- If $k == \text{left_size} + 1$, return root.
- If $k > \text{left_size} + 1$, go right with $k = k - \text{left_size} - 1$.

11. Complexity Analysis

Operation	Complexity	Notes
Insert	$O(\log n)$ expected	$O(n)$ worst-case (astronomically unlikely)
Delete	$O(\log n)$ expected	Same as insert
Search	$O(\log n)$ expected	Standard BST search
Kth Smallest	$O(\log n)$ expected	With subtree sizes
Rank	$O(\log n)$ expected	Count of elements $\leq \text{key}$
Split	$O(\log n)$ expected	Recursive split
Merge	$O(\log n)$ expected	Recursive merge
Space	$O(n)$	n nodes, each with 2 pointers + 3 fields

12. Common Patterns

Pattern 1: Ordered Set with Order Statistics

Identification: "Dynamic set where I need kth smallest and rank queries."

Approach: Treap with subtree sizes. Insert/delete in $O(\log n)$, kth smallest in $O(\log n)$.

Example: CSES (Order Statistics Set), Codeforces problems.

Pattern 2: Implicit Treap (Treap on Array)

Identification: "Range operations on an array: reverse, insert, delete, rotate."

Approach: Instead of BST key, use implicit index (size of left subtree). Split by position.

Example: CSES (Reversal Sort), Codeforces (Array Operations).

Pattern 3: Interval Union / Overlap Detection

Identification: "Maintain a set of intervals, support insert/delete/query."

Approach: Treap keyed by interval start. Store max end in subtree for overlap queries.

Example: Codeforces (Non-Decreasing Dilemma).

13. Common Mistakes

- **Forgetting to update subtree sizes:** After any rotation, split, or merge, `updateSize` must be called.
- **Not handling duplicates:** The basic treap doesn't handle duplicates. Either add a `count` field or use strict inequality in split.
- **Incorrect split condition:** `root->key <= key` for left treap ($\text{keys} \leq \text{key}$). For strict BST (no duplicates), use `<`.
- **Incorrect merge order:** All keys in left treap must be $\leq$ all keys in right treap. Violating this breaks the BST property.
- **Memory leaks:** In C++, deleting nodes requires proper cleanup. The `erase` function above merges children but doesn't delete the node when using split-merge approach.
- **Not seeding `rand()`:** Without `srand()`, priorities are the same every run.
- **Using `rand()` for high-priority applications:** Use `mt19937` for better randomness.
- **Stack overflow on deep recursion:** Treap is expected $O(\log n)$ depth, but worst-case recursion could be $O(n)$. Use iterative approach or increase stack size.

14. Edge Cases

- **Empty tree:** All operations should handle null root.
- **Single node:** Insert/delete/search should work correctly.
- **Duplicate keys:** Handle by storing count or using strict inequalities.
- **All keys inserted in order:** Treap still has $O(\log n)$ expected height due to random priorities.
- **All keys equal:** Only one node stored (or count incremented). Search works correctly.
- **Kth smallest with k out of bounds:** Return -1 or throw.
- **Deleting non-existent key:** The erase function should handle this gracefully (no-op).
- **Large number of nodes ($10^5 +$):** Works fine with $O(\log n)$ expected depth.

15. Variations

15.1 Implicit Treap

Instead of BST key, each node's position is implicit (determined by the size of its left subtree). Supports array operations: insert at position, delete at position, reverse range, rotate.

Use: When you need efficient range operations on a dynamic array.

15.2 Persistent Treap

Since split and merge are pure functions (they create new nodes), the treap can be made persistent easily. Each operation creates a new root without destroying the old one.

Use: When you need to access previous versions of the data structure.

15.3 Treap with Lazy Propagation

Add lazy tags to support range updates (add to range, reverse range, etc.).

Use: When you need both BST operations and range updates.

15.4 Randomized BST (without Treap)

Instead of rotations, use a different approach: during insertion, insert at the root with probability $1/(n+1)$ (using split+merge), otherwise insert recursively.

16. Related Algorithms/Data Structures

Algorithm	Connection	When to Choose
AVL Tree	Self-balancing BST, deterministic	When worst-case $O(\log n)$ is required
Red-Black Tree	Self-balancing BST, used in C++ <code>std::set</code>	Standard library, production code
Splay Tree	Self-adjusting BST	When recently accessed elements should be fast
Segment Tree	Range queries on static array	Static data, no insertions/deletions
Fenwick Tree	Prefix sums, simpler	Only prefix/suffix queries, no insertions

17. Practice Problems

Easy

1. **Search in a Binary Search Tree** — LeetCode 700 (Basic BST search, can use treap)
2. **Kth Largest Element in a Stream** — LeetCode 703 (Can use treap or heap)

Medium

3. **Kth Smallest Element in a BST** — LeetCode 230 (Order statistics, treap is overkill but works)
4. **My Calendar I** — LeetCode 729 (Interval scheduling with BST)
5. **Range Sum Query - Mutable** — LeetCode 307 (Segment tree / Fenwick, but treap can also work)

Hard

6. **Count of Smaller Numbers After Self** — LeetCode 315 (Treap with order statistics from right to left)
7. **Reverse Pairs** — LeetCode 493 (Treap with order statistics)
8. **My Calendar III** — LeetCode 732 (Treap with interval operations)

18. Interview Explanation

"A Treap is a randomized BST where each node has a random priority, and the tree maintains both the BST property (by key) and the heap property (by priority). The random priorities ensure the tree has $O(\log n)$ expected height for insert, delete, and search operations. The key advantage is its simplicity — the implementation is much shorter than AVL or Red-Black trees. With subtree sizes, it supports order statistics like kth smallest and rank in $O(\log n)$. The split and merge operations make it incredibly flexible — I can use an implicit treap for array operations. The expected performance is $O(\log n)$ for all operations, and the probability of a degenerate tree is negligible."

19. Revision Notes

- **Core idea:** BST + Heap with random priorities → expected $O(\log n)$ height
- **Node fields:** `key`, `priority`, `size`, `left`, `right`
- **Key operations:** `split`, `merge`, `rotateLeft`, `rotateRight`
- **Insert (rotation):** BST insert + rotate up if priority > parent
- **Insert (split):** `split(root, key)`, `merge(left, new_node, right)`
- **Delete (merge):** Find node, merge children, attach to parent
- **Kth smallest:** Use subtree sizes, compare with left size
- **Time:** $O(\log n)$ expected, $O(n)$ worst-case (unlikely)
- **Space:** $O(n)$
- **Common trap:** Forgetting to update sizes, not handling duplicates

20. Final Cheat Sheet

When to use	Main operations	Complexity	Key code idea	Edge cases
Ordered set with order statistics, dynamic set, split/merge	Insert, Delete, Search, Kth, Split, Merge	$O(\log n)$ expected all ops	Random priority, split/merge	Empty, duplicates, large n, recursion depth

MONTE CARLO ALGORITHM

1. Overview

A **Monte Carlo algorithm** is a randomized algorithm whose output may be **incorrect with a small probability**. The algorithm runs in **deterministic time** (always finishes within a given bound), but the correctness is probabilistic. By running the algorithm multiple times, the error probability can be reduced arbitrarily (to any desired level).

The name comes from the Monte Carlo Casino in Monaco, reflecting the algorithm's reliance on randomness.

2. Intuition

Imagine you are trying to estimate the area of an irregular shape. You could:

1. Draw a square around the shape.
2. Throw darts randomly at the square.
3. Count how many darts land inside the shape vs. total darts thrown.
4. $\text{Area} = (\text{inside} / \text{total}) \times \text{square area}.$

This is a Monte Carlo method — it uses randomness to estimate a value, and the estimate gets more accurate as you throw more darts. The answer is never exact, but you can make it arbitrarily close to the truth.

Analogy: A chef tasting a spoonful of soup to check if it's salty enough. One spoonful gives a rough estimate. More spoonfuls give a better estimate. But you never know with 100% certainty — you just reduce the probability of being wrong.

The key trade-off: Monte Carlo algorithms trade **deterministic correctness** for **speed**. They are fast (polynomial time) but may produce wrong answers. The error probability can be made exponentially small by repeating the algorithm.

3. When to Use It

- When you need fast approximate answers and exact answers are expensive
- When the problem is NP-hard or has no known polynomial-time exact algorithm
- When you need to estimate a numerical value (area, volume, integral, expectation)
- When you can trade correctness for speed
- In optimization (simulated annealing, genetic algorithms)
- In machine learning (random sampling, dropout, Monte Carlo Tree Search)
- In cryptography (primality testing with Miller-Rabin)
- When deterministic algorithms are too complex or slow

Common trigger phrases:

- "Approximate the value of..."
- "Randomized algorithm that may be wrong"
- "Probabilistic correctness"
- "Estimate with high probability"
- "Monte Carlo simulation"

4. When Not to Use It

- When you need guaranteed correct answers (use Las Vegas or deterministic algorithms)
- When the cost of error is catastrophic (medical, aviation, nuclear)
- When the problem has an efficient exact algorithm
- When you need to verify correctness in polynomial time (some problems have this property)
- When the input size is very small (deterministic algorithms are fast enough)
- When you cannot tolerate any error probability, no matter how small

5. Core Concepts

5.1 One-Sided vs Two-Sided Error

- **One-sided error:** The algorithm is only wrong in one direction. For example, a primality test might say "composite" (always correct) or "prime" (with some error probability).
- **Two-sided error:** The algorithm can be wrong in both directions.

Why it matters: One-sided error algorithms are easier to amplify (repeat the algorithm and take majority vote or AND/OR).

5.2 Error Probability

The probability that the algorithm gives a wrong answer. Usually denoted by δ (delta).

Why it matters: By repeating the algorithm $O(\log(1/\delta))$ times, the error probability can be reduced to δ .

5.3 Concentration Bounds (Chernoff, Hoeffding)

Mathematical tools that bound the probability that a random variable deviates from its expected value.

Why it matters: These bounds are used to prove that Monte Carlo algorithms have small error probabilities after enough repetitions.

5.4 Monte Carlo vs Las Vegas

Property	Monte Carlo	Las Vegas
Correctness	Probabilistic (may be wrong)	Always correct
Running time	Deterministic (always finishes)	Probabilistic (expected time)
Error amplification	Repeat and take majority	Not applicable
When to use	Need guaranteed time	Need guaranteed correctness

6. Step-by-Step Algorithm (Generic Monte Carlo)

1. Identify the problem and define a random process that simulates it.
2. Run the random process N times (N is chosen based on desired accuracy).
3. Aggregate the results (average, count, majority vote, etc.).
4. Return the aggregated result with a confidence bound.

Example: Estimating π using Monte Carlo:

1. Generate N random points (x, y) in the square $[-1, 1] \times [-1, 1]$.
2. Count how many points satisfy $x^2 + y^2 \leq 1$ (inside the unit circle).
3. $\pi \approx 4 \times (\text{inside} / \text{total})$.

7. Dry Run

Estimating π with $N = 1000$ points:

Trial	Points Inside	Total Points	Estimate = $4 \times (\text{inside}/\text{total})$
1	785	1000	3.140

Trial	Points Inside	Total Points	Estimate = $4 \times (\text{inside}/\text{total})$
2	791	1000	3.164
3	778	1000	3.112
4	788	1000	3.152
5	793	1000	3.172
Average	787	1000	3.148

True $\pi = 3.14159\dots$ The estimate gets closer as N increases.

Example: Miller-Rabin Primality Test (Monte Carlo)

Test if $n = 221$ is prime:

Iteration	Base a	Result	Meaning
1	2	Inconclusive (may be prime)	221 passes test for $a=2$
2	3	221 is composite	$221 = 13 \times 17$
3	5	221 is composite	Confirmed

Miller-Rabin has one-sided error: if it says "composite," it's always correct. If it says "prime," it may be wrong with probability $\leq 1/4$.

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

// ----- 1. ESTIMATE  $\pi$  USING MONTE CARLO -----
double estimatePi(int numPoints) {
    srand(time(0));
    int inside = 0;

    for (int i = 0; i < numPoints; i++) {
        double x = (double)rand() / RAND_MAX * 2.0 - 1.0; // [-1, 1]
        double y = (double)rand() / RAND_MAX * 2.0 - 1.0; // [-1, 1]

        if (x * x + y * y <= 1.0) {
            inside++;
        }
    }

    return 4.0 * inside / numPoints;
}

// ----- 2. MILLER-RABIN PRIMALITY TEST (Monte Carlo) -----
using ll = long long;
```

```
// Modular multiplication (prevents overflow for large numbers)
ll modMul(ll a, ll b, ll mod) {
    ll res = 0;
    a %= mod;
    while (b > 0) {
        if (b & 1) res = (res + a) % mod;
        a = (a * 2) % mod;
        b >>= 1;
    }
    return res;
}

// Modular exponentiation
ll modPow(ll base, ll exp, ll mod) {
    ll res = 1;
    base %= mod;
    while (exp > 0) {
        if (exp & 1) res = modMul(res, base, mod);
        base = modMul(base, base, mod);
        exp >>= 1;
    }
    return res;
}

// Miller-Rabin test for a specific base
bool millerTest(ll d, ll n, ll a) {
    ll x = modPow(a, d, n);
    if (x == 1 || x == n - 1) return true;

    while (d != n - 1) {
        x = modMul(x, x, n);
        d <<= 1;
        if (x == n - 1) return true;
        if (x == 1) return false;
    }
    return false;
}

// Miller-Rabin primality test
// One-sided error: if returns false, n is definitely composite
// If returns true, n is probably prime (error  $\leq 1/4^k$ )
bool isPrimeMillerRabin(ll n, int k = 5) {
    if (n < 2) return false;
    if (n == 2 || n == 3) return true;
    if (n % 2 == 0) return false;

    // Write n-1 as d * 2^s
    ll d = n - 1;
    while (d % 2 == 0) d /= 2;

    // Witness loop
    for (int i = 0; i < k; i++) {
        ll a = 2 + rand() % (n - 4);
        if (!millerTest(d, n, a)) return false;
    }
}
```

```

    }
    return true;
}

// Deterministic Miller-Rabin for 64-bit integers
bool isPrimeDeterministic(ll n) {
    if (n < 2) return false;
    if (n == 2 || n == 3) return true;
    if (n % 2 == 0) return false;

    ll d = n - 1;
    while (d % 2 == 0) d /= 2;

    // These bases are sufficient for n < 2^64
    vector<ll> bases = {2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37};
    for (ll a : bases) {
        if (a >= n) break;
        if (!millerTest(d, n, a)) return false;
    }
    return true;
}

// ----- 3. MONTE CARLO INTEGRATION -----
// Estimate  $\int_0^1 f(x) dx$ 
double monteCarloIntegral(double (*f)(double), double low, double high, int
numPoints) {
    srand(time(0));
    double sum = 0.0;

    for (int i = 0; i < numPoints; i++) {
        double x = low + (double)rand() / RAND_MAX * (high - low);
        sum += f(x);
    }

    return (high - low) * sum / numPoints;
}

double sampleFunction(double x) {
    return x * x; //  $\int_0^1 x^2 dx = 1/3 \approx 0.333$ 
}

// ----- DEMO -----
int main() {
    srand(time(0));

    cout << "Monte Carlo Estimation of  $\pi$ :" << endl;
    cout << "N=1000:  $\pi \approx$  " << estimatePi(1000) << endl;
    cout << "N=10000:  $\pi \approx$  " << estimatePi(10000) << endl;
    cout << "N=100000:  $\pi \approx$  " << estimatePi(100000) << endl;
    cout << "N=1000000:  $\pi \approx$  " << estimatePi(1000000) << endl;
    cout << "True  $\pi$ : " << M_PI << endl;

    cout << "\nPrimality Testing (Miller-Rabin):" << endl;
    cout << "Is 17 prime? " << (isPrimeMillerRabin(17) ? "Yes" : "No") << endl;
}

```

```

    cout << "Is 221 prime? " << (isPrimeMillerRabin(221) ? "Yes" : "No") << endl;
    cout << "Is 1000000007 prime? " << (isPrimeMillerRabin(1000000007) ? "Yes" :
    "No") << endl;

    cout << "\nMonte Carlo Integration:" << endl;
    cout << " $\int_0^1 x^2 dx \approx$  " << monteCarloIntegral(sampleFunction, 0, 1, 100000) <<
    endl;
    cout << "Exact:      " << 1.0/3.0 << endl;

    return 0;
}

```

9. Python Implementation

```

import random
import math

# ----- 1. ESTIMATE  $\pi$  -----
def estimate_pi(num_points):
    inside = 0
    for _ in range(num_points):
        x = random.uniform(-1, 1)
        y = random.uniform(-1, 1)
        if x*x + y*y <= 1:
            inside += 1
    return 4.0 * inside / num_points

# ----- 2. MILLER-RABIN PRIMALITY TEST -----
def mod_pow(base, exp, mod):
    result = 1
    base %= mod
    while exp > 0:
        if exp & 1:
            result = (result * base) % mod
        base = (base * base) % mod
        exp >>= 1
    return result

def miller_test(d, n, a):
    x = mod_pow(a, d, n)
    if x == 1 or x == n - 1:
        return True
    while d != n - 1:
        x = (x * x) % n
        d <<= 1
        if x == n - 1:
            return True
        if x == 1:
            return False
    return False

```

```

def is_prime_miller_rabin(n, k=5):
    if n < 2:
        return False
    if n == 2 or n == 3:
        return True
    if n % 2 == 0:
        return False

    d = n - 1
    while d % 2 == 0:
        d //= 2

    for _ in range(k):
        a = random.randint(2, n - 2)
        if not miller_test(d, n, a):
            return False
    return True

# ----- 3. MONTE CARLO INTEGRATION -----
def monte_carlo_integral(f, low, high, num_points):
    total = 0.0
    for _ in range(num_points):
        x = random.uniform(low, high)
        total += f(x)
    return (high - low) * total / num_points

# Demo
print("Monte Carlo Estimation of  $\pi$ :")
for n in [1000, 10000, 100000, 1000000]:
    print(f"N={n}:  $\pi \approx \{\text{estimate\_pi}(n)\}$ ")
print(f"True  $\pi$ : {math.pi}")

print("\nPrimality Testing:")
print(f"Is 17 prime? {is_prime_miller_rabin(17)}")
print(f"Is 221 prime? {is_prime_miller_rabin(221)}")
print(f"Is 1000000007 prime? {is_prime_miller_rabin(1000000007)}")

print("\nMonte Carlo Integration:")
def f(x): return x*x
print(f" $\int_0^1 x^2 dx \approx \{\text{monte\_carlo\_integral}(f, 0, 1, 100000)\}$ ")
print(f"Exact: {1/3}")

```

10. Code Explanation

Estimating π :

- Generate N random points uniformly in the square $[-1, 1] \times [-1, 1]$.
- Count points inside the unit circle ($x^2 + y^2 \leq 1$).
- Area of square = 4, area of circle = π . Ratio = $\pi/4$.
- $\pi \approx 4 \times (\text{inside} / \text{total})$.

Miller-Rabin Primality Test:

- Based on the fact that for prime n , the equation $a^2 \equiv 1 \pmod{n}$ has only solutions $a \equiv \pm 1 \pmod{n}$.
- Write $n-1 = d \times 2^s$. For a random base a , compute $a^d \pmod{n}$. If it's ± 1 , n passes. Otherwise, repeatedly square up to s times.
- If n is composite, at least $3/4$ of bases will detect it (one-sided error).
- For 64-bit integers, a fixed set of deterministic bases works perfectly.

Monte Carlo Integration:

- Generate random points uniformly in the integration range.
- Average the function values at these points.
- Multiply by the range width to get the integral estimate.
- Error decreases as $O(1/\sqrt{N})$ (standard deviation of the mean).

11. Complexity Analysis

Algorithm	Time Complexity	Error Rate	Notes
π estimation	$O(N)$	$O(1/\sqrt{N})$	N is number of points
Miller-Rabin	$O(k \log^3 n)$	$O(1/4^k)$	k is number of iterations
Monte Carlo integration	$O(N)$	$O(1/\sqrt{N})$	N is number of points
Error amplification	$O(k \times T)$	$O(\delta^k)$	Repeat k times, T is base time

12. Common Patterns

Pattern 1: Numerical Estimation (π , Integrals, etc.)

Identification: "Estimate the value of π / area / integral."

Approach: Random sampling + averaging.

Example: Estimate π , compute area under curve.

Pattern 2: Randomized Primality Testing

Identification: "Check if a large number is prime."

Approach: Miller-Rabin, Fermat primality test.

Example: LeetCode, cryptography, competitive programming.

Pattern 3: Monte Carlo Tree Search (MCTS)

Identification: "Find the best move in a game (Go, Chess, etc.)."

Approach: Simulation-based tree search with random rollouts.

Example: AlphaGo, game AI.

Pattern 4: Randomized Optimization (Simulated Annealing)

Identification: "Find approximate solution to NP-hard optimization problem."

Approach: Random walk with decreasing temperature, accept worse solutions with probability.

Example: Traveling salesman, graph partitioning.

13. Common Mistakes

- **Assuming Monte Carlo gives exact answers:** It gives approximations with probabilistic guarantees.
- **Not running enough iterations:** Error decreases as $O(1/\sqrt{N})$. To halve the error, you need 4× more samples.
- **Using biased random sampling:** The random numbers must be uniformly distributed over the sample space.
- **Not understanding the error probability:** Miller-Rabin with $k = 1$ has error $\leq 1/4$. With $k = 5$, error $\leq 1/4^5 = 1/1024$.
- **Forgetting to seed the RNG:** Without seeding, results are deterministic and not actually random.
- **Using Monte Carlo when a deterministic algorithm exists:** Use the exact algorithm when available.
- **Not handling edge cases in primality testing:** $n = 1, 2, 3$, even numbers must be handled separately.

14. Edge Cases

- **π estimation with $N = 0$:** Division by zero. Handle separately.
- **π estimation with $N = 1$:** Very inaccurate. Return 0 or 4 depending on the single point.
- **Miller-Rabin with $n = 1$:** Return false (not prime).
- **Miller-Rabin with $n = 2$:** Return true.
- **Miller-Rabin with n even:** Return false immediately.
- **Monte Carlo integration with no points:** Division by zero.
- **Error amplification:** Running the algorithm k times reduces error to δ^k , but this is only valid for independent repetitions.

15. Variations

15.1 Las Vegas → Monte Carlo

Some Las Vegas algorithms can be converted to Monte Carlo by stopping after a fixed time and returning a best guess.

15.2 Monte Carlo Tree Search (MCTS)

Used in game AI (AlphaGo, AlphaZero). Combines tree search with random rollouts.

15.3 Markov Chain Monte Carlo (MCMC)

Used for sampling from complex probability distributions. Metropolis-Hastings, Gibbs sampling.

15.4 Randomized Rounding

Used in approximation algorithms. Solve a linear programming relaxation, then randomly round the solution to an integer solution.

16. Related Algorithms/Data Structures

Algorithm	Connection	When to Choose
Las Vegas	Randomized, always correct	When correctness is critical
Deterministic	Always correct, fixed time	When exact answer is needed
Approximation Algorithm	Gives approximate answer with guarantee	Often deterministic, gives worst-case bound
Randomized Rounding	Monte Carlo for combinatorial optimization	When LP relaxation has fractional solution
Simulated Annealing	Monte Carlo for optimization	For NP-hard problems with no known PTAS

17. Practice Problems

Easy

1. **Generate Random Point in a Circle** — LeetCode 478 (Rejection sampling, a Monte Carlo technique)
2. **Estimate the Value of π** — Various platforms (Basic Monte Carlo simulation)

Medium

3. **Random Pick with Weight** — LeetCode 528 (Monte Carlo: uses random sampling)
4. **Implement Rand10() Using Rand7()** — LeetCode 470 (Rejection sampling, Monte Carlo)

Hard

5. **Random Flip Matrix** — LeetCode 519 (Monte Carlo with rejection sampling)
6. **Probability of a Knight's Move** — LeetCode 688 (Dynamic programming, but can be solved with Monte Carlo simulation)

18. Interview Explanation

"A Monte Carlo algorithm is a randomized algorithm that always finishes in a fixed time but may produce an incorrect answer with some probability. The classic example is estimating π by throwing random points at a square and counting how many fall inside the inscribed circle. The error probability can be made arbitrarily small by repeating the algorithm. For example, Miller-Rabin primality test is a Monte Carlo algorithm — if it says 'composite,' it's always correct (one-sided error). If it says 'prime,' there's at most 25% chance of error per iteration, but with 5 iterations, the error drops below 0.1%. Monte Carlo is useful when exact solutions are too expensive and approximate answers are acceptable."

19. Revision Notes

- **Core idea:** Randomized algorithm, always finishes, may be wrong (probabilistically)
- **One-sided error:** Only wrong in one direction (e.g., Miller-Rabin: says "prime" but it's composite)
- **Two-sided error:** Can be wrong both ways

- **Error amplification:** Repeat k times, error drops to δ^k
- **π estimation:** $4 \times (\text{points inside circle} / \text{total points})$
- **Miller-Rabin:** $O(k \log^3 n)$, error $\leq 1/4^k$
- **Monte Carlo integration:** $(\text{high} - \text{low}) \times \text{average}(f(x))$ over random points
- **Error rate:** $O(1/\sqrt{N})$ for sampling-based methods
- **Key trade-off:** Correctness vs. Time

20. Final Cheat Sheet

When to use	Main operations	Complexity	Key code idea	Edge cases
Fast approximate solutions, NP-hard problems, numerical estimation	Random sampling, Counting, Averaging	Varies by problem, often $O(N)$	Random sampling + aggregation	$N=0$, error probability, bias in sampling

LAS VEGAS ALGORITHM

1. Overview

A **Las Vegas algorithm** is a randomized algorithm that **always produces a correct result**, but the **running time is a random variable**. The algorithm may take a long time on some runs (even infinite, in theory), but its expected running time is bounded.

The name comes from Las Vegas, Nevada — reflecting the idea that the algorithm "gambles" with time, not correctness.

2. Intuition

Imagine you are looking for a lost key in a dark room. You can search randomly, and you will eventually find the key (always correct), but the time it takes depends on your luck.

- **Deterministic approach:** Search systematically row by row → guaranteed $O(\text{area})$ time.
- **Las Vegas approach:** Search randomly, checking random spots → expected $O(\text{area})$ time, but sometimes faster, sometimes slower.

Analogy: A chef who keeps tasting a dish until it's perfect. They always get the right result (correctness is guaranteed), but the time varies (sometimes it's perfect on the first try, sometimes it takes multiple attempts).

The key trade-off: Las Vegas algorithms trade **deterministic running time** for **simplicity and expected efficiency**. They are often simpler to implement and have good expected performance on typical inputs.

3. When to Use It

- When you need guaranteed correct answers (unlike Monte Carlo)
- When the worst-case running time is bad but the average case is good
- When you want a simple randomized algorithm instead of a complex deterministic one
- When the input distribution is adversarial (randomization spreads the risk)

- When you can restart the algorithm if it takes too long
- In randomized data structures (treap, skip list)
- In randomized sorting (randomized quicksort)
- In randomized selection (quickselect)

Common trigger phrases:

- "Always correct, but expected time is..."
- "Randomized algorithm with guaranteed correctness"
- "Las Vegas algorithm"
- "Expected $O(n)$ time, worst-case $O(n^2)$ "

4. When Not to Use It

- When you need worst-case running time guarantees (use deterministic algorithms)
- When the algorithm's expected time is not better than the deterministic alternative
- When the probability of a long run is unacceptable (e.g., real-time systems)
- When the problem has an efficient deterministic algorithm
- When you need to guarantee completion by a deadline (use Monte Carlo instead)
- When the algorithm can get stuck in infinite loops (handle with a cutoff)

5. Core Concepts

5.1 Expected Running Time

The average running time over all possible random choices. This is the key metric for Las Vegas algorithms.

Why it matters: A Las Vegas algorithm is considered efficient if its expected running time is polynomial, even if the worst-case is exponential.

5.2 Las Vegas vs Monte Carlo

Property	Las Vegas	Monte Carlo
Correctness	Always correct	May be wrong (probabilistic)
Running time	Probabilistic (expected time)	Deterministic (always finishes)
Error	No error	Small error probability
Amplification	Not needed	Repeat to reduce error
Conversion	Convert to MC by stopping early	Can't convert to LV without error

5.3 Conversion: Las Vegas $\rightarrow$ Monte Carlo

By stopping a Las Vegas algorithm after a fixed time T and returning a best guess, you get a Monte Carlo algorithm. The error probability is the probability that the Las Vegas algorithm takes longer than T .

Why it matters: This is a common technique in practice. If you can't wait for the worst case, stop early and accept a small error probability.

5.4 Randomization for Avoiding Worst-Case Inputs

The key insight: by making random choices, the algorithm avoids being "stuck" on any specific input. The worst-case input is no worse than any other input in expectation.

6. Step-by-Step Algorithm (Generic Las Vegas)

1. Repeat until a correct result is found: a. Make a random choice or generate a random candidate. b. Check if the candidate is correct. c. If correct, return it. d. If not, try again.

Example: Randomized Quickselect (find kth smallest):

1. Pick a random pivot from the array.
2. Partition around the pivot.
3. If pivot is the kth element, return it.
4. If $k < \text{pivot index}$, recurse on left subarray.
5. If $k > \text{pivot index}$, recurse on right subarray.
6. Always correct (deterministic partition), expected $O(n)$ time.

7. Dry Run

Randomized Quickselect: Find the 4th smallest element in [7, 2, 1, 6, 8, 5, 3, 4]

Step	Subarray	Random Pivot	Partition Result	k	Action
1	[7, 2, 1, 6, 8, 5, 3, 4]	index 4 (value 8)	pivot at index 7	4	$k < \text{pivot index}$, go left
2	[7, 2, 1, 6, 5, 3, 4]	index 2 (value 1)	pivot at index 0	4	$k > \text{pivot index}$, go right
3	[7, 2, 6, 5, 3, 4]	index 3 (value 5)	pivot at index 4	3	$k < \text{pivot index}$, go left
4	[7, 2, 6, 3, 4]	index 1 (value 2)	pivot at index 1	3	$k > \text{pivot index}$, go right
5	[7, 6, 3, 4]	index 0 (value 7)	pivot at index 3	2	$k < \text{pivot index}$, go left
6	[6, 3, 4]	index 1 (value 3)	pivot at index 1	2	$k > \text{pivot index}$, go right
7	[6, 4]	index 0 (value 6)	pivot at index 1	1	$k < \text{pivot index}$, go left
8	[4]	—	single element	1	Return 4

Result: 4th smallest element = 4. Always correct, expected $O(n)$ time.

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

// ----- 1. RANDOMIZED QUICKSORT (Las Vegas) -----
// Always correct, expected  $O(n \log n)$  time

int partition(vector<int>& arr, int low, int high) {
    int pivot = arr[high];
```

```

    int i = low;
    for (int j = low; j < high; j++) {
        if (arr[j] <= pivot) {
            swap(arr[i], arr[j]);
            i++;
        }
    }
    swap(arr[i], arr[high]);
    return i;
}

int randomPartition(vector<int>& arr, int low, int high) {
    int r = low + rand() % (high - low + 1);
    swap(arr[r], arr[high]);
    return partition(arr, low, high);
}

void quickSort(vector<int>& arr, int low, int high) {
    if (low < high) {
        int pi = randomPartition(arr, low, high);
        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}

// ----- 2. RANDOMIZED QUICKSELECT (Las Vegas) -----
// Always correct, expected O(n) time

int quickSelect(vector<int>& arr, int low, int high, int k) {
    if (low == high) return arr[low];

    int pi = randomPartition(arr, low, high);
    int leftSize = pi - low + 1; // number of elements in left partition

    if (k == leftSize) return arr[pi];
    if (k < leftSize) return quickSelect(arr, low, pi - 1, k);
    return quickSelect(arr, pi + 1, high, k - leftSize);
}

int kthSmallest(vector<int> arr, int k) {
    if (k < 1 || k > arr.size()) return -1; // invalid
    return quickSelect(arr, 0, arr.size() - 1, k);
}

// ----- 3. RANDOMIZED BINARY SEARCH WITH RANDOMIZED PIVOT -----
// (Not a standard algorithm, but demonstrates the Las Vegas idea)
// Expected O(log n), worst-case O(n)

int randomizedBinarySearch(const vector<int>& arr, int target) {
    int n = arr.size();
    int low = 0, high = n - 1;

    while (low <= high) {
        // Pick a random index in [low, high] instead of mid

```

```

        int mid = low + rand() % (high - low + 1);

        if (arr[mid] == target) return mid;
        if (arr[mid] < target) low = mid + 1;
        else high = mid - 1;
    }
    return -1;
}

// ----- 4. RANDOMIZED TRIAL DIVISION (for factoring small numbers) -----
-
// Always correct, expected time depends on input

vector<int> getPrimeFactors(int n) {
    vector<int> factors;
    while (n % 2 == 0) { factors.push_back(2); n /= 2; }
    while (n % 3 == 0) { factors.push_back(3); n /= 3; }

    // Randomize the order of trial division
    for (int i = 5; i * i <= n; i += 2) {
        // With probability 1/2, try i or i+2 first
        int step = rand() % 2 == 0 ? 0 : 2;
        int d1 = i + step;
        int d2 = i + (step == 0 ? 2 : 0);

        while (n % d1 == 0) { factors.push_back(d1); n /= d1; }
        while (n % d2 == 0) { factors.push_back(d2); n /= d2; }
    }
    if (n > 1) factors.push_back(n);
    return factors;
}

// ----- DEMO -----
int main() {
    srand(time(0));

    // Quickselect
    vector<int> arr = {7, 2, 1, 6, 8, 5, 3, 4};
    cout << "4th smallest: " << kthSmallest(arr, 4) << endl; // 4
    cout << "1st smallest: " << kthSmallest(arr, 1) << endl; // 1
    cout << "8th smallest: " << kthSmallest(arr, 8) << endl; // 8

    // Quicksort
    vector<int> arr2 = {7, 2, 1, 6, 8, 5, 3, 4};
    quickSort(arr2, 0, arr2.size() - 1);
    cout << "Sorted: ";
    for (int x : arr2) cout << x << " ";
    cout << endl;

    return 0;
}

```

9. Python Implementation

```
import random

# ----- 1. RANDOMIZED QUICKSORT -----
def partition(arr, low, high):
    pivot = arr[high]
    i = low
    for j in range(low, high):
        if arr[j] <= pivot:
            arr[i], arr[j] = arr[j], arr[i]
            i += 1
    arr[i], arr[high] = arr[high], arr[i]
    return i

def random_partition(arr, low, high):
    r = random.randint(low, high)
    arr[r], arr[high] = arr[high], arr[r]
    return partition(arr, low, high)

def quicksort(arr, low, high):
    if low < high:
        pi = random_partition(arr, low, high)
        quicksort(arr, low, pi - 1)
        quicksort(arr, pi + 1, high)

# ----- 2. RANDOMIZED QUICKSELECT -----
def quickselect(arr, low, high, k):
    if low == high:
        return arr[low]

    pi = random_partition(arr, low, high)
    left_size = pi - low + 1

    if k == left_size:
        return arr[pi]
    if k < left_size:
        return quickselect(arr, low, pi - 1, k)
    return quickselect(arr, pi + 1, high, k - left_size)

def kth_smallest(arr, k):
    if k < 1 or k > len(arr):
        return None
    # Make a copy to avoid modifying original
    arr_copy = arr.copy()
    return quickselect(arr_copy, 0, len(arr_copy) - 1, k)

# Demo
arr = [7, 2, 1, 6, 8, 5, 3, 4]
print(f"4th smallest: {kth_smallest(arr, 4)}") # 4
print(f"1st smallest: {kth_smallest(arr, 1)}") # 1
print(f"8th smallest: {kth_smallest(arr, 8)}") # 8
```

```
arr2 = [7, 2, 1, 6, 8, 5, 3, 4]
quicksort(arr2, 0, len(arr2) - 1)
print(f"Sorted: {arr2}")
```

10. Code Explanation

Randomized Quickselect:

- Same `partition` and `randomPartition` as randomized quicksort.
- `quickSelect` checks if the pivot is the k th element. If yes, return it. Otherwise, recurse on the appropriate side.
- `leftSize = pi - low + 1` gives the number of elements from `low` to `pi` (inclusive).
- If $k == \text{leftSize}$, the pivot is the k th element.
- If $k < \text{leftSize}$, the k th element is in the left subarray.
- If $k > \text{leftSize}$, the k th element is in the right subarray. Adjust k .

Why it's Las Vegas:

- The algorithm always returns the correct k th smallest element.
- The running time is a random variable, with expected $O(n)$ and worst-case $O(n^2)$.
- The partition is deterministic — the randomness is only in the pivot selection.

11. Complexity Analysis

Algorithm	Expected Time	Worst-Case Time	Space	Notes
Randomized Quicksort	$O(n \log n)$	$O(n^2)$	$O(\log n)$	Probability of $O(n^2)$ is negligible
Randomized Quickselect	$O(n)$	$O(n^2)$	$O(\log n)$	Expected $O(n)$ for any k
Randomized Treap	$O(\log n)$	$O(n)$	$O(n)$	Expected $O(\log n)$ per operation
Randomized Skip List	$O(\log n)$	$O(n)$	$O(n \log n)$	Expected $O(\log n)$ per operation

12. Common Patterns

Pattern 1: Randomized Quickselect

Identification: "Find the k th smallest/largest element."

Approach: Random pivot + partition + recurse on one side.

Example: LeetCode 215 (Kth Largest Element in an Array)

Pattern 2: Randomized Quicksort

Identification: "Sort an array efficiently."

Approach: Random pivot + partition + recurse on both sides.

Example: LeetCode 912 (Sort an Array)

Pattern 3: Randomized Data Structures

Identification: "Build a balanced BST / skip list."

Approach: Use random priorities (treap) or random levels (skip list).

Example: CSES (Order Statistics Set)

Pattern 4: Randomized Algorithm with Restart

Identification: "Find a solution by random trials until success."

Approach: Generate random candidates, check correctness, repeat until found.

Example: Randomized 2-SAT (Papadimitriou's algorithm), randomized min-cut.

13. Common Mistakes

- **Confusing Las Vegas with Monte Carlo:** Las Vegas is always correct but has variable time. Monte Carlo is sometimes wrong but has fixed time.
- **Not handling the worst-case running time:** In practice, if the worst-case is too slow (e.g., $O(n^2)$ for quicksort on $n = 10^6$), use a hybrid approach (like Introsort).
- **Forgetting that expected time is not guaranteed:** The algorithm could run longer than expected, though with low probability.
- **Not using a good random number generator:** `rand()` has limited range and bias. Use `mt19937` for serious applications.
- **Not handling the case where the algorithm could loop forever:** In theory, a Las Vegas algorithm could run forever. In practice, add a cutoff or iteration limit.

14. Edge Cases

- **Quickselect with $k = 0$ or $k > n$:** Invalid input. Return error.
- **Quickselect with single element:** Return the element.
- **Quickselect with all equal elements:** Always returns the correct element (all are equal).
- **Quicksort with empty array:** No-op.
- **Quicksort with single element:** No-op.
- **Quicksort with all equal elements:** With Lomuto partition, pivot index is always `high`. This is $O(n^2)$ even with random pivot if all elements are equal. Use three-way partition.

15. Variations

15.1 Las Vegas → Monte Carlo Conversion

By stopping a Las Vegas algorithm after a fixed time T and returning a best guess, you get a Monte Carlo algorithm. The error probability = $P(T_{LV} > T)$.

Use: When you need guaranteed completion time.

15.2 Randomized Incremental Construction

Used in computational geometry (Delaunay triangulation, convex hull). Add elements in random order, always correct, expected $O(n \log n)$.

Use: Geometric algorithms.

15.3 Randomized Min-Cut (Karger's Algorithm)

A Las Vegas algorithm for finding the minimum cut in a graph. Randomly contract edges until 2 vertices remain, repeat. Always correct, expected $O(n^2)$ time.

Use: Graph theory, network reliability.

16. Related Algorithms/Data Structures

Algorithm	Connection	When to Choose
Monte Carlo	Randomized, may be wrong, fixed time	When guaranteed time is needed
Deterministic	Always correct, fixed time	When both are needed
Randomized Quicksort	Classic Las Vegas	Sorting
Randomized Quickselect	Classic Las Vegas	Order statistics
Treap	Las Vegas data structure	Balanced BST with order statistics

17. Practice Problems

Easy

1. **Kth Largest Element in an Array** — LeetCode 215 (Quickselect, Las Vegas)
2. **Sort an Array** — LeetCode 912 (Randomized Quicksort, Las Vegas)

Medium

3. **K Closest Points to Origin** — LeetCode 973 (Quickselect variant)
4. **Top K Frequent Elements** — LeetCode 347 (Quickselect on frequency array)

Hard

5. **Median of Two Sorted Arrays** — LeetCode 4 (Generalized kth element, can use quickselect idea)
6. **Kth Smallest Element in a Sorted Matrix** — LeetCode 378 (Binary search approach, but quickselect is alternative)

18. Interview Explanation

"A Las Vegas algorithm is a randomized algorithm that always produces the correct answer but has a variable running time. The classic example is randomized quicksort — it always sorts correctly, but the running time depends on the random pivot choices. The expected time is $O(n \log n)$, but the worst case

is $O(n^2)$ — though the probability of hitting the worst case is astronomically small. Another example is randomized quickselect for finding the k th smallest element, which has expected $O(n)$ time. The key advantage of Las Vegas algorithms is that they combine the simplicity of randomization with the guarantee of correctness. I would use them when I need guaranteed correctness but want to avoid complex deterministic algorithms."

19. Revision Notes

- **Core idea:** Always correct, expected running time is bounded
- **Key difference from Monte Carlo:** Correctness is guaranteed, time is random
- **Randomized Quicksort:** Expected $O(n \log n)$, worst $O(n^2)$
- **Randomized Quickselect:** Expected $O(n)$, worst $O(n^2)$
- **Conversion to MC:** Stop after fixed time $\rightarrow$ may be wrong
- **Expected vs worst-case:** Expected is the average over random choices
- **Common examples:** Quicksort, Quickselect, Treap, Skip List, Karger's min-cut
- **Key trade-off:** Simplicity + expected efficiency vs. complex deterministic algorithms

20. Final Cheat Sheet

When to use	Main operations	Complexity	Key code idea	Edge cases
Need guaranteed correctness, want simple randomized algorithm	Random pivot + partition + recurse	Expected $O(n \log n)$ or $O(n)$, worst $O(n^2)$	Random pivot, always return correct result	All equal elements, empty, single, k out of bounds

FINAL COMPARISON: MONTE CARLO vs LAS VEGAS

Aspect	Monte Carlo	Las Vegas
Correctness	May be wrong (probabilistic)	Always correct
Running time	Deterministic (always finishes)	Probabilistic (expected time)
Error probability	Small (can be made arbitrarily small)	Zero
Error amplification	Repeat and take majority	Not applicable
Conversion	Cannot be converted to Las Vegas	Can be converted to Monte Carlo (stop early)
When to use	Need guaranteed time, can tolerate small error	Need guaranteed correctness, can tolerate variable time
Examples	Miller-Rabin (primality), π estimation, Monte Carlo integration	Randomized Quicksort, Quickselect, Treap, Skip List

COMPREHENSIVE CHEAT SHEET: RANDOMIZED ALGORITHMS

Algorithm	Type	When to Use	Time	Space	Key Idea
Randomized Quicksort	Las Vegas	Sorting large arrays, unknown input distribution	$O(n \log n)$ expected	$O(\log n)$	Random pivot → expected balance
Reservoir Sampling	Las Vegas	Random sample from stream of unknown size	$O(n)$	$O(k)$	Replace with probability k/i
Fisher-Yates Shuffle	Las Vegas	Random permutation, random sample without replacement	$O(n)$	$O(1)$	Pick random from unshuffled portion
Random Pick with Weight	Monte Carlo	Weighted random selection	$O(n)$ prep, $O(\log n)$ pick	$O(n)$	Prefix sums + binary search
Treap	Las Vegas	Balanced BST with order statistics, split/merge	$O(\log n)$ expected	$O(n)$	Random priority → heap property
Monte Carlo (general)	Monte Carlo	Fast approximate solutions, numerical estimation	$O(N)$	$O(1)$	Random sampling + aggregation
Las Vegas (general)	Las Vegas	Guaranteed correctness, simple randomization	Expected polynomial	Varies	Random choices, always correct

This guide covers the essential randomized algorithms for SDE placements, online assessments, and competitive programming. Each algorithm is presented with intuition, implementation, complexity analysis, and practical tips for interview success.